

```
/* ===== 文件: src/models/vibnet.py ===== */
```

```
"""整体模型装配：空频双分支轻量 VIB-Net
```

```
数据流（对应申报书 图表2 技术路线）：
```

```
输入 x (B,3,224,224)
```

```
├── 空域分支 SpatialBranch          -> F_spa (B,896,14,14) + phi(x) (B,768)
```

```
└── 频域分支 FrequencyBranch        -> F_freq (B,256,14,14)
```

```
↓
```

```
CS-CAM 交叉注意力融合              -> F_fusion (B,512,14,14)          [1.1(3)]
```

```
↓
```

```
GS1 梯度停止层
```

```
GS2 梯度停止层
```

```
[1.3(1)]
```

```
↓ 分层 VIB（仅分类分支）
```

```
↓ 定位支路 Mobile-UNetv2 + 边缘头
```

```
[1.2 / 1.3(3)]
```

```
↓ MLP 分类头 256→128→2
```

```
↓ 224x224 篡改掩码 + 边缘图
```

```
↓ 整图真伪概率 y_cls
```

```
↓ 像素级定位 M_pred
```

```
损失 (1.4) : L_total = Σ 0.5*exp(-s_i)*L'_i + 0.5*Σ s_i
```

```
"""
```

```
from __future__ import annotations
```

```
from typing import Dict, Iterable, List, Optional
```

```
import torch
```

```
import torch.nn as nn
```

```
from .cs_cam import CSCAM
```

```
from .freq_branch import FrequencyBranch
```

```
from .gradient_stop import GradientStopLayer
```

```
from .mobile_unetv2 import LocalizationBranch
```

```
from .spatial_branch import SpatialBranch
```

```
from .vib import HierarchicalVIB
```

```
# 可冻结/解冻的模块组名
```

```
MODULE_GROUPS = ("spatial", "freq", "fusion", "vib", "cls_head", "localization")
```

```
class VIBNet(nn.Module):
```

```
    def __init__(self, cfg: dict):
```

```
        super().__init__()
```

```
        self.cfg = cfg
```

```
        m, fcfg, lcfg = cfg["model"], cfg["model"]["fusion"], cfg["model"]["localization"]
```

```
        # ---- 1.1 空频双分支
```

```
        self.spatial = SpatialBranch(cfg)
```

```
        self.freq = FrequencyBranch(cfg)
```

```
        # ---- 1.1(3) 交叉注意力融合
```

```
        self.fusion = CSCAM(
```

```
            spa_dim=self.spatial.out_dim,
```

```
            freq_dim=self.freq.out_dim,
```

```
            dim=fcfg.get("dim", 512),
```

```
            use_cross_attention=fcfg.get("use_cross_attention", True),
```

```
        )
```

```
        fusion_dim = fcfg.get("dim", 512)
```

```
        # ---- 1.3(1) 梯度停止层
```

```
        use_gs = lcfg.get("use_grad_stop", True)
```

```
        self.gs_cls = GradientStopLayer(stop=lcfg.get("gs_stop_cls", False), name="GS1_cls")
```

```
        self.gs_loc = GradientStopLayer(stop=lcfg.get("gs_stop_loc", False), name="GS2_loc")
```

```
        if not use_gs:      # 消融：移除梯度隔离
```

```

        self.gs_cls.stop = self.gs_loc.stop = False
    self.use_grad_stop = use_gs
    # ---- 1.2 分层 VIB (仅分类支路最后一层)
    vcfg = m["vib"]
    self.vib = HierarchicalVIB(
        in_dim=fusion_dim,
        hidden_dim=vcfg.get("hidden_dim", 256),
        latent_dim=vcfg.get("latent_dim", 256),
        kl_clip_min=vcfg.get("kl_clip_min", 0.0),
        kl_clip_max=vcfg.get("kl_clip_max", 10.0),
        kl_reduction=vcfg.get("kl_reduction", "mean"),
        kl_clip_grad_through=vcfg.get("kl_clip_grad_through", True),
    )
    self.use_vib = vcfg.get("use_vib", True)
    # ---- 分类头: MLP 256 -> 128 -> 2 (公式 27)
    h = m["head"]
    latent_dim = vcfg.get("latent_dim", 256)
    in_dim_cls = latent_dim if self.use_vib else fusion_dim
    self.cls_head = nn.Sequential(
        nn.Linear(in_dim_cls, h.get("cls_hidden", 128)),
        nn.ReLU(inplace=True),
        nn.Dropout(0.2),
        nn.Linear(h.get("cls_hidden", 128), h.get("num_classes", 2)),
    )
    # ---- 1.3(3) 定位支路
    self.localization = LocalizationBranch(cfg)
# =====
def forward(
    self,
    x: torch.Tensor,
    sample_vib: bool = True,
    beta: float = 0.0,
    return_intermediate: bool = False,
) -> Dict[str, torch.Tensor]:
    # 1.1 空频双分支
    f_spa, cls_token = self.spatial(x)          # (B,896,14,14), (B,768)
    f_freq = self.freq(x)                      # (B,256,14,14)
    # 1.1(3) 融合
    f_fusion = self.fusion(f_spa, f_freq)       # (B,512,14,14)
    # ---- 分类支路 (经 GS1 + VIB)
    f_cls_in = self.gs_cls(f_fusion)
    vib_out = self.vib(f_cls_in, sample=sample_vib, beta=beta)
    z = vib_out["z"] if self.use_vib else f_cls_in.mean(dim=(2, 3))
    cls_logits = self.cls_head(z)              # (B,2)
    # ---- 定位支路 (经 GS2, 保留细粒度)
    f_loc_in = self.gs_loc(f_fusion)
    loc_out = self.localization(f_loc_in)
    out: Dict[str, torch.Tensor] = {
        "cls_logits": cls_logits,
        "cls_prob": torch.softmax(cls_logits, dim=-1),
        "mask_logits": loc_out["mask_logits"],
        "mask_prob": loc_out["mask_prob"],
    }

```

```

        "kl": vib_out["kl"],
        "kl_raw": vib_out["kl_raw"],
        "mu": vib_out["mu"],
        "sigma": vib_out["sigma"],
        "beta": torch.tensor(float(beta)),
    }
    if "edge_logits" in loc_out:
        out["edge_logits"] = loc_out["edge_logits"]
        out["edge_prob"] = loc_out["edge_prob"]
    if return_intermediate:
        out.update({
            "F_spa": f_spa, "F_freq": f_freq, "F_fusion": f_fusion,
            "freq_mask": self.freq.mask_net(
                torch.loglp(self.freq.to_amp_phase(x)[0])
            ),
            "cls_token": cls_token, "latent_z": z,
        })
    return out
# =====
# 参数管理：按模块组冻结 / 解冻（支撑三阶段训练）
# =====
def set_requires_grad(self, groups: Iterable[str], requires_grad: bool) -> None:
    groups = list(groups)
    if "all" in groups:
        for p in self.parameters():
            p.requires_grad_(requires_grad)
        return
    for g in groups:
        if g not in MODULE_GROUPS:
            raise ValueError(f"未知模块组 {g}，可选：{MODULE_GROUPS}")
        module = getattr(self, g)
        for p in module.parameters():
            p.requires_grad_(requires_grad)
def apply_stage(self, freeze: List[str], train: List[str]) -> None:
    """按阶段配置设置 requires_grad。train 优先于 freeze."""
    self.set_requires_grad(["all"], True)
    if freeze:
        self.set_requires_grad(freeze, False)
    if train and "all" not in train:
        self.set_requires_grad(["all"], False)
        self.set_requires_grad(train, True)
def trainable_parameters(self):
    return [p for p in self.parameters() if p.requires_grad]
# =====
def set_dft_mode(self, mode: str) -> None:
    """切换频域分支的 DFT 实现。
    * "torch" —— 训练阶段用，GPU 上 FFT 更快；
    * "matmul" —— **导出 ONNX 前必须切换**，否则会因
        `aten::fft_fft2` 不受 opset 17 支持而导出失败。
    两种实现数值等价（误差 ~1e-5），详见 src/models/dft.py。
    """
    self.freq.set_dft_mode(mode)

```

```

# =====
def summary(self) -> Dict[str, float]:
    def n(mod):
        return sum(p.numel() for p in mod.parameters()) / 1e6
    total = sum(p.numel() for p in self.parameters()) / 1e6
    trainable = sum(p.numel() for p in self.parameters() if p.requires_grad) / 1e6
    info = {
        "spatial(M)": n(self.spatial),
        "freq(M)": n(self.freq),
        "fusion(M)": n(self.fusion),
        "vib(M)": n(self.vib),
        "cls_head(M)": n(self.cls_head),
        "localization(M)": n(self.localization),
        "total(M)": total,
        "trainable(M)": trainable,
        "fp32_size(MB)": total * 4,
    }
    return info
@torch.no_grad()
def sanity_check(self, batch: int = 2) -> Dict[str, object]:
    """端端维度自检: 核对申报书各公式的输出维度。"""
    was_training = self.training
    self.eval()
    size = self.cfg["data"]["image_size"]
    x = torch.randn(batch, 3, size, size)
    out = self.forward(x, sample_vib=False, beta=0.1, return_intermediate=True)
    self.train(was_training)
    return {
        "输入 x": tuple(x.shape),
        "phi(x)/cls_token": tuple(out["cls_token"].shape),
        "F_spa (公式3)": tuple(out["F_spa"].shape),
        "F_freq (公式12)": tuple(out["F_freq"].shape),
        "F_fusion (公式17)": tuple(out["F_fusion"].shape),
        "latent z (公式23)": tuple(out["latent_z"].shape),
        "cls_logits (公式27)": tuple(out["cls_logits"].shape),
        "mask_prob (公式28)": tuple(out["mask_prob"].shape),
        "edge_prob (公式29)": tuple(out.get("edge_prob", torch.empty(0)).shape),
        "kl (公式25)": float(out["kl"]),
        "参数量": self.summary(),
    }
# =====
def build_model(cfg: dict) -> VIBNet:
    """工厂函数: 从配置字典构建模型。"""
    model = VIBNet(cfg)
    return model
def load_config(path: str) -> dict:
    """读取 YAML 配置, 并把相对路径解析为项目根目录下的绝对路径。"""
    import os
    import yaml
    with open(path, "r", encoding="utf-8") as f:
        cfg = yaml.safe_load(f)
    root = os.path.dirname(os.path.dirname(os.path.abspath(path)))

```

```

    for key in ("output_dir", "ckpt_dir"):
        if key in cfg.get("project", {}):
            cfg["project"][key] = os.path.join(root, cfg["project"][key])
    if "data" in cfg and "root" in cfg["data"] and not os.path.isabs(cfg["data"]["root"]):
        cfg["data"]["root"] = os.path.join(root, cfg["data"]["root"])
    if "data" in cfg and "demo" in cfg["data"] and "root" in cfg["data"]["demo"]:
        if not os.path.isabs(cfg["data"]["demo"]["root"]):
            cfg["data"]["demo"]["root"] = os.path.join(root, cfg["data"]["demo"]["root"])
    return cfg
/* ===== 文件: src/models/spatial_branch.py ===== */
"""1.1 (1) 空域特征提取分支 —— "全局语义特征 + 局部纹理特征"
对应申报书公式 (1)(2)(3):
    phi(x)   = CLIP-ViT-B/16(x)[: , 0, :]           in R^768      (1) 全局语义 (CLS token)
    F_local  = GAP(DS-Conv3(x))                     in R^128      (2) 3 层深度可分离卷积
    F_spa    = Concat(phi(x), F_local)                in R^896      (3) 通道拼接

```

实现说明（重要，写论文/答辩时会用到）

公式 (1)(3) 给出的是 **\*\*向量\*\*** 形式，但 1.3(3) 定位支路要求

"512 维空频融合特征图（尺寸为 14x14x512）"。二者必须统一，本实现的做法是：

- \* 全局语义：取 CLIP-ViT-B/16 的 patch token（去掉 CLS token）→ (B, 196, 768)  
→ 重排为 (B, 768, 14, 14) 的空间特征图；CLS token 单独保留，  
作为整图真伪判别的"全局语义向量"（与公式 (1) 一致）。
- \* 局部纹理：DS-Conv3 直接在 224x224 原图上做，输出 (B, 128, 224, 224)，  
自适应平均池化到 14x14 → (B, 128, 14, 14)，与全局特征空间对齐。
- \* 通道拼接：(B, 896, 14, 14) 即 F\_spa 的 "特征图" 形式；  
对 (B, 896, 14, 14) 做 GAP 即得到公式 (3) 的 896 维向量。  
验证：`model.spatial\_branch.sanity\_check()`。

这样既满足公式 (1)(2)(3)，又满足 1.3(3) 对空间分辨率的要求。

"""

```

from __future__ import annotations
import os
from typing import Dict, List, Optional, Tuple
import torch
import torch.nn as nn
import torch.nn.functional as F
# =====
# 深度可分离卷积
# =====
class DepthwiseSeparableConv(nn.Module):
    """深度卷积 + 逐点卷积，3x3 / stride 1 / pad 1（申报书 1.1(1) 局部纹理特征提取）。"""
    def __init__(self, in_ch: int, out_ch: int, kernel: int = 3, stride: int = 1, padding: int = 1):
        super().__init__()
        self.depthwise = nn.Conv2d(
            in_ch, in_ch, kernel, stride, padding, groups=in_ch, bias=False
        )
        self.pointwise = nn.Conv2d(in_ch, out_ch, 1, 1, 0, bias=False)
        self.bn = nn.BatchNorm2d(out_ch)
        self.act = nn.ReLU(inplace=True)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.act(self.bn(self.pointwise(self.depthwise(x))))
class LocalTextureNet(nn.Module):

```

"""3 层深度可分离卷积网络：通道 32 -> 64 -> 128，输出空间特征图。

对 224x224 输入，三层 stride=1 保持分辨率，最后池化到 14x14。

（等价于公式 (2) 的 DS-Conv3；GAP 得到 128 维向量。）

"""

```
def __init__(self, in_ch: int = 3, channels: List[int] = (32, 64, 128), out_hw: int = 14):
```

```
    super().__init__()
```

```
    chs = [in_ch] + list(channels)
```

```
    self.blocks = nn.Sequential(
```

```
        *[(
```

```
            DepthwiseSeparableConv(chs[i], chs[i + 1], kernel=3, stride=1, padding=1)
```

```
            for i in range(len(channels))
```

```
        ]
```

```
    )
```

```
    self.out_hw = out_hw
```

```
    self.out_dim = channels[-1]
```

```
def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
    f = self.blocks(x) # (B, 128, H, W)
```

```
    return F.adaptive_avg_pool2d(f, self.out_hw) # (B, 128, 14, 14)
```

```
# -----
```

```
# CLIP-ViT-B/16 骨干
```

```
# -----
```

```
class CLIPViTBackbone(nn.Module):
```

"""CLIP-ViT-B/16: 冻结前 6 层，仅微调后 6 层（申报书 1.1(1)）。

- 优先从本地目录加载权重（离线场景），其次从 HuggingFace 名称加载；

- 若两者都不可用，可退回 `TinyViTFallback`（结构同构的轻量随机初始化骨干），  
用于无网络/无 GPU 环境下跑通全流程（指标不可用于论文，仅用于验证代码正确性）。

"""

```
def __init__(
```

```
    self,
```

```
    pretrained_name: str = "openai/clip-vit-base-patch16",
```

```
    local_dir: str | None = None,
```

```
    freeze_first_n: int = 6,
```

```
    unfreeze_last_n: int = 6,
```

```
    allow_fallback: bool = True,
```

```
    force_fallback: bool = False,
```

```
    variant: str = "b16",
```

```
):
```

```
    super().__init__()
```

```
    self.variant = variant if variant in BACKBONE_PRESETS else "b16"
```

```
    hidden, depth, heads, mlp_ratio = BACKBONE_PRESETS[self.variant]
```

```
    self.preset = dict(hidden=hidden, depth=depth, heads=heads, mlp_ratio=mlp_ratio)
```

```
    self.hidden_dim = hidden
```

```
    self.grid = 14 # 224 / 16 = 14
```

```
    self.num_patches = self.grid * self.grid # 196
```

```
    self.is_fallback = False
```

```
    self.freeze_first_n = freeze_first_n
```

```
    self.unfreeze_last_n = unfreeze_last_n
```

```
    clip = None
```

```
    if not force_fallback and self.variant == "b16":
```

```
        clip = self._load_clip(pretrained_name, local_dir, allow_fallback)
```

```
    if clip is None:
```

```
        # 未取得预训练权重：按档位构造离线骨干（b16 档同样走这里，保证可导出 ONNX）
```

```

        self.clip = TinyViTFallback(patch=16, img=224, **self.preset)
        self.is_fallback = True
    else:
        self.clip = clip
    self._apply_freeze_policy()
# -----
@staticmethod
def _load_clip(name: str, local_dir: str | None, allow_fallback: bool):
    try:
        from transformers import CLIPVisionModel # 延迟导入, 避免强依赖
    except Exception:
        if not allow_fallback:
            raise
        print("[CLIPViTBackbone] 未安装 transformers, 使用 TinyViT 兜底骨干。")
        return None
    # 先尝试"只用本地缓存/本地目录", 避免无网络时卡在下载上;
    # 若本地没有权重, 且显式设置 VIB_NET_ALLOW_DOWNLOAD=1, 才联网下载。
    attempts = []
    if local_dir:
        attempts.append((local_dir, True))
    attempts.append((name, True))
    if os.environ.get("VIB_NET_ALLOW_DOWNLOAD") == "1":
        attempts.append((name, False))
    for ckpt, local_only in attempts:
        try:
            return CLIPVisionModel.from_pretrained(ckpt, local_files_only=local_only)
        except Exception as e: # noqa: BLE001
            print(f"[CLIPViTBackbone] 加载 {ckpt} (local_only={local_only}) 失败: "
                  f"{type(e).__name__}")
    print("[CLIPViTBackbone] 未取得 CLIP 预训练权重。"
          "如需联网下载: 先 export VIB_NET_ALLOW_DOWNLOAD=1; "
          "或把权重放到 clip_local_dir; 或用 backbone: tiny_vit 跑通流程。")
    if allow_fallback:
        print("[CLIPViTBackbone] 使用 TinyViT 兜底骨干 (仅用于流程验证, 指标无意义)。")
        return None
    raise RuntimeError("无法加载 CLIP 权重")
# -----
def _apply_freeze_policy(self) -> None:
    """冻结前 freeze_first_n 层, 解冻后 unfreeze_last_n 层 (含 embedding 之外的层)。"""
    if self.is_fallback:
        for p in self.clip.parameters():
            p.requires_grad_(True)
        return
    # 先全部冻结
    for p in self.clip.parameters():
        p.requires_grad_(False)
    layers = self.clip.vision_model.encoder.layers
    total = len(layers)
    n_unfreeze = min(self.unfreeze_last_n, total)
    for layer in layers[total - n_unfreeze:]:
        for p in layer.parameters():
            p.requires_grad_(True)

```

```

# 后置 LayerNorm 与视觉投影头参与微调
if hasattr(self.clip.vision_model, "post_layernorm"):
    for p in self.clip.vision_model.post_layernorm.parameters():
        p.requires_grad_(True)
if getattr(self.clip, "vision_model", None) is not None and hasattr(
    self.clip.vision_model, "pre_layrnorm"
):
    for p in self.clip.vision_model.pre_layrnorm.parameters():
        p.requires_grad_(True)
n_train = sum(p.numel() for p in self.parameters() if p.requires_grad)
n_total = sum(p.numel() for p in self.parameters())
print(
    f"[CLIPViTBackbone] 冻结前 {total - n_unfreeze}/{total} 层, "
    f"可训练参数 {n_train/1e6:.2f}M / {n_total/1e6:.2f}M"
)

# -----
def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """返回 (全局语义向量 cls (B,768), patch token 特征图 (B,768,14,14))。"""
    out = self.clip(pixel_values=x)
    tokens = out.last_hidden_state          # (B, 1+N, 768)
    cls_token = tokens[:, 0, :]             # 公式 (1):  $\phi(x)$  in  $\mathbb{R}^{768}$ 
    patch_tokens = tokens[:, 1:, :]         # (B, 196, 768)
    b, n, c = patch_tokens.shape
    g = int(n ** 0.5)
    fmap = patch_tokens.transpose(1, 2).reshape(b, c, g, g)  # (B, 768, 14, 14)
    return cls_token, fmap

class _ManualTransformerBlock(nn.Module):
    """手写 Transformer 编码块 (pre-LN)。
    为什么不直接用 `nn.TransformerEncoder`：
    它的 `aten::_transformer_encoder_layer_fwd` 是**融合算子**，
    `torch.onnx.export` 在 opset 17 下无法导出（报 `UnsupportedOperatorError`）。
    而申报书 3.1 明确要求导出 ONNX opset 17，因此这里用最基础的
    matmul / softmax / layer_norm 手写一遍，保证整条部署链路可用。
    """
    def __init__(self, dim: int = 768, heads: int = 8, mlp_ratio: float = 2.0):
        super().__init__()
        self.norm1 = nn.LayerNorm(dim)
        self.qkv = nn.Linear(dim, dim * 3)
        self.proj = nn.Linear(dim, dim)
        self.norm2 = nn.LayerNorm(dim)
        self.mlp = nn.Sequential(
            nn.Linear(dim, int(dim * mlp_ratio)),
            nn.GELU(),
            nn.Linear(int(dim * mlp_ratio), dim),
        )
        self.heads = heads
        self.scale = (dim // heads) ** -0.5

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        b, n, c = x.shape
        h = self.heads
        y = self.norm1(x)
        qkv = self.qkv(y).reshape(b, n, 3, h, c // h).permute(2, 0, 3, 1, 4)

```

```

        q, k, v = qkv[0], qkv[1], qkv[2]                                # (B,h,N,d)
        attn = torch.softmax((q * self.scale) @ k.transpose(-1, -2), dim=-1)
        y = (attn @ v).transpose(1, 2).reshape(b, n, c)
        x = x + self.proj(y)
        x = x + self.mlp(self.norm2(x))

    return x
# 骨干规模档位预设 (hidden, depth, heads, mlp_ratio)
#  b16 —— 申报书原始设定, 对应 CLIP-ViT-B/16 结构, 87.5M 参数
#  s16 —— ViT-S/16 规模 (384 维), 22.5M 参数, 精度/体积折中
#  ti16 —— ViT-Ti/16 规模 (192 维), 5.9M 参数, **离线可构造、体积最小**
# 注意: 预训练权重缺失时这些档位是随机初始化的, 只用于「跑通流程 + 报告体积」,
# 真正用于精度实验请把对应预训练权重放到 clip_local_dir。
BACKBONE_PRESETS: Dict[str, Tuple[int, int, int, float]] = {
    "b16": (768, 12, 12, 4.0),
    "s16": (384, 12, 6, 4.0),
    "ti16": (192, 12, 3, 4.0),
    "ti8": (192, 8, 3, 4.0),
    # ↓ 仅用于加载"引入档位概念之前"训练出的旧 checkpoint (768 维 / 6 层 / mlp 2.0)。
    # 不建议用于新训练: 它不是任何真实 ViT 的规模, 只是为了旧权重还能被加载和复现。
    "legacy_vit768_6": (768, 6, 8, 2.0),
}

# 仅用于兼容旧权重的档位, 切换时会额外提示
LEGACY_VARIANTS = frozenset({"legacy_vit768_6"})

class TinyViTFallback(nn.Module):
    """离线可构造的 ViT 骨干: 输出接口与 CLIP-ViT-B/16 完全一致 (CLS + N patch tokens)。
    两个用途:
    1. **兜底**: 无网络 / 无预训练权重时让整条流水线可跑通;
    2. **轻量档骨干**: `hidden/depth/heads` 可调, 用于在「模型体积 ≤ 120MB」
        这一硬指标下做体积-精度权衡 (见 docs/09)。
    刻意只用 matmul / softmax / layer_norm 等基础算子, 保证可导出 ONNX opset 17。
    """
    def __init__(self, hidden: int = 768, depth: int = 6, patch: int = 16, img: int = 224,
                  heads: int = 8, mlp_ratio: float = 2.0):
        super().__init__()
        self.patch_embed = nn.Conv2d(3, hidden, patch, patch)
        n = (img // patch) ** 2
        self.cls_token = nn.Parameter(torch.zeros(1, 1, hidden))
        self.pos_embed = nn.Parameter(torch.zeros(1, n + 1, hidden))
        nn.init.trunc_normal_(self.cls_token, std=0.02)
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        self.blocks = nn.ModuleList(
            [ManualTransformerBlock(hidden, heads=heads, mlp_ratio=mlp_ratio)
             for _ in range(depth)]
        )
        self.norm = nn.LayerNorm(hidden)

    def forward(self, pixel_values: torch.Tensor):
        b = pixel_values.shape[0]
        x = self.patch_embed(pixel_values).flatten(2).transpose(1, 2)  # (B,N,768)
        x = torch.cat([self.cls_token.expand(b, -1, -1), x], dim=1) + self.pos_embed
        for blk in self.blocks:
            x = blk(x)
        x = self.norm(x)

```

```

class _Out:
    def __init__(self, last_hidden_state):
        self.last_hidden_state = last_hidden_state
    return _Out(x)
# -----
# 空域分支整体
# -----
class SpatialBranch(nn.Module):
    """空域分支:  $F_{spa} = \text{Concat}(\phi(x), F_{local}) \in \mathbb{R}^{896}$  (空间形式 (B,896,14,14))。"""
    def __init__(self, cfg: dict):
        super().__init__()
        s = cfg["model"]["spatial"]
        backbone = str(s.get("backbone", "clip_vit_b16")).lower()
        self.backbone = CLIPViTBackbone(
            pretrained_name=s.get("clip_pretrained", "openai/clip-vit-base-patch16"),
            local_dir=s.get("clip_local_dir"),
            freeze_first_n=s.get("freeze_first_n_layers", 6),
            unfreeze_last_n=s.get("unfreeze_last_n_layers", 6),
            allow_fallback=True,
            force_fallback=backbone in ("tiny_vit", "fallback", "tiny"),
            variant=str(s.get("backbone_variant", "b16")).lower(),
        )
        self.local = LocalTextureNet(
            in_ch=3,
            channels=s.get("local_channels", [32, 64, 128]),
            out_hw=14,
        )
        self.global_dim = self.backbone.hidden_dim # 768
        self.local_dim = s.get("local_dim", 128) # 128
        self.out_dim = self.global_dim + self.local_dim # 896
    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        """返回 ( $F_{spa}$  特征图 (B,896,14,14), 全局语义向量 (B,768))。"""
        cls_vec, patch_map = self.backbone(x) # (B,768), (B,768,14,14)
        local_map = self.local(x) # (B,128,14,14)
        f_spa = torch.cat([patch_map, local_map], dim=1) # (B,896,14,14) 公式(3)
        return f_spa, cls_vec
# -----
@torch.no_grad()
def sanity_check(self, img_size: int = 224) -> dict:
    """核对与公式 (1) (2) (3) 的维度一致性。"""
    self.eval()
    x = torch.randn(2, 3, img_size, img_size)
    f_spa, cls_vec = self.forward(x)
    gap_vec = f_spa.mean(dim=(2, 3))
    info = {
        "phi(x) 维度": tuple(cls_vec.shape),
        "F_local 维度": (cls_vec.shape[0], self.local_dim),
        "F_spa 特征图": tuple(f_spa.shape),
        "F_spa(GAP) 向量维度": tuple(gap_vec.shape),
        "期望": "(2,768) + (2,128) -> (2,896,14,14)",
    }
    return info

```

```

# -----
# checkpoint 兼容性: 从权重反推骨干档位
# -----
def infer_backbone_variant(state_dict: dict) -> Optional[str]:
    """从 checkpoint 的 state_dict 反推出它用的是哪个骨干档位。
    为什么需要这个函数
    -----
    骨干档位在引入 `BACKBONE_PRESETS` 前后不兼容:
    * 旧版兜底骨干 = 768 维 / 6 层 / mlp_ratio 2.0
    * 现在 b16 档 = 768 维 / 12 层 / mlp_ratio 4.0
    用旧 checkpoint 配新 `default.yaml` 直接 `load_state_dict` 会抛出
    `RuntimeError: size mismatch for spatial.backbone.clip.blocks.0.mlp.0.weight`
    —— 只看这个报错很难猜到是“骨干档位变了”。
    本函数按 **MLP 隐层宽度** 与 **Transformer 层数** 反推档位,
    返回档位名 (`BACKBONE_PRESETS` 的 key), 认不出来则返回 None。
    """

    import re
    probe_keys = (
        "spatial.backbone.clip.blocks.0.mlp.0.weight",
        "spatial.backbone.blocks.0.mlp.0.weight",
    )

    probe = next((k for k in probe_keys if k in state_dict), None)
    if probe is None:
        return None
    shape = tuple(state_dict[probe].shape)
    if len(shape) < 2:
        return None
    inter, hidden = int(shape[0]), int(shape[1])
    idx = set()
    for k in state_dict:
        m = re.match(r"spatial\.backbone\.(?:clip\.)?blocks\.(\d+)\.", k)
        if m:
            idx.add(int(m.group(1)))
    depth = len(idx)
    if depth == 0:
        return None
    ratio = inter / max(hidden, 1)
    for name, (h, d, _heads, r) in BACKBONE_PRESETS.items():
        if h == hidden and d == depth and abs(r - ratio) < 1e-6:
            return name
    return None

def align_cfg_to_checkpoint(cfg: dict, state_dict: dict, verbose: bool = True) -> Optional[str]:
    """按 checkpoint 的骨干档位就地修正 cfg, 返回推断出的档位名。
    这样 `evaluate.py` / `train.py --resume` / 桌面工具在加载旧权重时
    不需要用户手工去改 YAML, 也不会因为尺寸不匹配而崩溃。
    """

    variant = infer_backbone_variant(state_dict)
    if variant is None:
        return None
    cur = str(cfg["model"]["spatial"].get("backbone_variant", "b16")).lower()
    if variant == cur:
        return variant

```

```

if verbose:
    print(f"[ckpt] 检测到 checkpoint 的骨干档位为 '{variant}', "
          f"与配置里的 '{cur}' 不一致, 已自动对齐。"
          f"（如需保持一致, 请把 YAML 里的 backbone_variant 改为 '{variant}'）")
    if variant in LEGACY_VARIANTS:
        print("[ckpt] □ 该档位是「引入档位概念之前」的旧兜底骨干（768/6层/mlp2.0）, "
              "仅用于复现旧结果。它不对应任何真实 ViT 规模、也没有预训练权重, "
              "其精度指标**不能写进论文**。正式实验请用 b16/s16 档重新训练。")
    cfg["model"]["spatial"]["backbone_variant"] = variant
    # 档位不是 b16 时一定是离线构造的骨干, 避免又去联网找 CLIP 权重
    if variant != "b16":
        cfg["model"]["spatial"]["backbone"] = "tiny_vit"
    return variant
/* ===== 文件: src/models/freq_branch.py ===== */
"""1.1 (2) 频域特征提取分支 —— 幅值-相位联合表征 + 可学习频域掩码
对应申报书公式 (4)~(12):

$$F(u,v) = \text{DFT}(I(x,y)) \quad (4)$$


$$A(u,v) = |F_{\text{shift}}(u,v)| \quad P(u,v) = \arg(F_{\text{shift}}) \quad (5) (6)$$


$$M(u,v) = \text{sigmoid}(\text{Conv2}(\text{Conv1}(A))) \quad \text{in } [0,1] \quad (7) \quad \text{可学习频域掩码}$$


$$A' = A \odot M, P' = P \odot M \quad (8) (9)$$


$$F_{\text{amp}} = \text{GAP}(\text{CNN}_{\text{amp}}(A')) \quad \text{in } R^{128} \quad (10)$$


$$F_{\text{phase}} = \text{GAP}(\text{CNN}_{\text{phase}}(P')) \quad \text{in } R^{128} \quad (11)$$


$$F_{\text{freq}} = \text{Concat}(F_{\text{amp}}, F_{\text{phase}}) \quad \text{in } R^{256} \quad (12)$$


```

实现要点（工程细节，直接影响能否复现）

1. 输入先转灰度（3 通道平均），保证 DFT 得到单通道二维频谱；  
若直接用 RGB，则幅值/相位需要分别对 3 通道处理，参数量  $\times 3$ ，工程上不划算。
2. 2D-DFT 等价于 `torch.fft.fft2(x, norm='ortho')`，移频用 `torch.fft.fftshift`。
3. 幅值谱动态范围极大（对数尺度上  $1e^{-3} \sim 1e^3$ ），必须先做对数压缩  
`log(1+A)` 再送入掩码网络，否则 Sigmoid 掩码会被少数高频点主导。
4. 相位 `[-pi, pi]` 直接送入网络；为便于 OpenVINO 导出，用  
`atan2(sin, cos)` 而不是 `angle`（前者算子更通用，且避免复数类型）。  
实现上：`phase = torch.atan2(imag, real)`，全程只用实数张量。
5. 掩码网络只有 2 层、通道  $1 \rightarrow 32 \rightarrow 1$ ，输出  $(B, 1, 224, 224)$ ，广播到 A、P 上。
6. 双路特征提取网络把  $224 \times 224$  下采样到  $14 \times 14$  (stride 4 + 4)，保证与  
空域分支空间尺寸一致；对  $(B, 128, 14, 14)$  做 GAP 即得公式 (10)(11) 的 128 维向量。

消融开关（对应申报书消融实验）：

```

use_phase=False      -> 仅用幅值, F_freq in R^128 (验证"频域信息利用率提升 100%")
use_learnable_mask=False -> 用固定中频掩码替代可学习掩码
"""

```

```

from __future__ import annotations
from typing import List, Tuple
import torch
import torch.nn as nn
import torch.nn.functional as F
from .dft import DFT2D
# -----

```

```

class LearnableFrequencyMask(nn.Module):

```

```

    """2 层卷积生成自适应频域掩码（公式 7）。

```

```

    Conv(1->32, 5x5, s1, p2) -> ReLU -> Conv(32->1, 5x5, s1, p2) -> Sigmoid
    """

```

```

def __init__(self, hidden: int = 32, kernel: int = 5):
    super().__init__()
    pad = kernel // 2
    self.conv1 = nn.Conv2d(1, hidden, kernel, 1, pad)
    self.bn1 = nn.BatchNorm2d(hidden)
    self.conv2 = nn.Conv2d(hidden, 1, kernel, 1, pad)
    self.act = nn.ReLU(inplace=True)
def forward(self, log_amp: torch.Tensor) -> torch.Tensor:
    m = self.conv2(self.act(self.bn1(self.conv1(log_amp))))
    return torch.sigmoid(m)          # (B,1,H,W) ∈ [0,1]
class FixedBandMask(nn.Module):
    """固定中频掩码（消融基线）：保留归一化半径  $r \in [0.15, 0.55]$  的环形频带。"""
    def __init__(self, low: float = 0.15, high: float = 0.55):
        super().__init__()
        self.low, self.high = low, high
    def forward(self, log_amp: torch.Tensor) -> torch.Tensor:
        b, _, h, w = log_amp.shape
        yy = torch.linspace(-1, 1, h, device=log_amp.device).view(-1, 1).expand(h, w)
        xx = torch.linspace(-1, 1, w, device=log_amp.device).view(1, -1).expand(h, w)
        r = torch.sqrt(xx ** 2 + yy ** 2)
        m = ((r >= self.low) & (r <= self.high)).float()
        return m.view(1, 1, h, w).expand(b, 1, h, w)
class FreqPathNet(nn.Module):
    """单路频域特征提取：2 层 3x3 卷积 32→128，两次 stride=2 下采样到 14x14。
    对应公式 (10) (11) 中 CNN_amp / CNN_phase（结构相同）。
    """
    def __init__(self, in_ch: int = 1, channels: List[int] = (32, 128), out_hw: int = 14):
        super().__init__()
        chs = [in_ch] + list(channels)
        layers = []
        for i in range(len(channels)):
            layers += [
                nn.Conv2d(chs[i], chs[i + 1], 3, 1, 1, bias=False),
                nn.BatchNorm2d(chs[i + 1]),
                nn.ReLU(inplace=True),
            ]
        self.conv = nn.Sequential(*layers)
        self.out_hw = out_hw
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        f = self.conv(x)
        return F.adaptive_avg_pool2d(f, self.out_hw)      # (B,128,14,14)
# -----
class FrequencyBranch(nn.Module):
    def __init__(self, cfg: dict):
        super().__init__()
        f = cfg["model"]["freq"]
        self.use_phase = f.get("use_phase", True)
        self.use_learnable_mask = f.get("use_learnable_mask", True)
        hidden = f.get("mask_hidden", 32)
        kernel = f.get("mask_kernel", 5)
        chs = f.get("feat_channels", [32, 128])
        # 2D-DFT: 训练用 torch FFT（最快），导出前切到 matmul（ONNX 可导出）

```

```

        size = cfg["data"].get("image_size", 224)
        self.dft = DFT2D(size, size, mode=f.get("dft_mode", "torch"))
        self.mask_net = (
            LearnableFrequencyMask(hidden, kernel) if self.use_learnable_mask else FixedBandMask()
        )
        self.amp_net = FreqPathNet(1, chs, out_hw=14)
        self.amp_dim = chs[-1]
        if self.use_phase:
            self.phase_net = FreqPathNet(1, chs, out_hw=14)
            self.phase_dim = chs[-1]
        else:
            self.phase_net = None
            self.phase_dim = 0
        self.out_dim = self.amp_dim + self.phase_dim    # 256 (或 128)
# -----
def set_dft_mode(self, mode: str) -> None:
    """切换 DFT 实现: 'torch' (训练) / 'matmul' (导出)。"""
    self.dft.mode = mode
def to_amp_phase(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """公式 (4)(5)(6): 2D-DFT -> fftshift -> 分离幅值谱 / 相位谱 (全实数运算)。
    输入先转灰度 (3 通道平均), 保证得到单通道二维频谱。
    """
    gray = x.mean(dim=1, keepdim=True)                # (B,1,H,W) 转灰度
    return self.dft(gray)
def forward(self, x: torch.Tensor) -> torch.Tensor:
    amp, phase = self.to_amp_phase(x)                  # (B,1,224,224)
    log_amp = torch.log1p(amp)                          # 动态范围压缩
    mask = self.mask_net(log_amp)                      # (B,1,224,224)  $\in [0,1]$  公式(7)
    amp_enh = amp * mask                                # 公式 (8)
    amp_map = self.amp_net(amp_enh)                    # (B,128,14,14)
    if self.use_phase:
        phase_enh = phase * mask                      # 公式 (9)
        phase_map = self.phase_net(phase_enh)          # (B,128,14,14)
        f_freq = torch.cat([amp_map, phase_map], dim=1) # (B,256,14,14) 公式(12)
    else:
        f_freq = amp_map                              # (B,128,14,14)
    return f_freq
# -----
@torch.no_grad()
def forward_with_debug(self, x: torch.Tensor) -> dict:
    amp, phase = self.to_amp_phase(x)
    log_amp = torch.log1p(amp)
    mask = self.mask_net(log_amp)
    out = {
        "amp": amp, "phase": phase, "mask": mask,
        "F_freq": self.forward(x),
    }
    return out
@torch.no_grad()
def sanity_check(self, img_size: int = 224) -> dict:
    self.eval()
    x = torch.randn(2, 3, img_size, img_size)

```

```

    amp, phase = self.to_amp_phase(x)
    f = self.forward(x)
    return {
        "A(u,v)": tuple(amp.shape),
        "P(u,v)": tuple(phase.shape),
        "F_freq 特征图": tuple(f.shape),
        "F_freq(GAP) 向量维度": tuple(f.mean(dim=(2, 3)).shape),
        "use_phase": self.use_phase,
        "use_learnable_mask": self.use_learnable_mask,
    }
/* ===== 文件: src/models/dft.py ===== */
"""可导出的 2D-DFT 实现 (申报书公式 (4) (5) (6))

```

为什么需要这个模块 (这是一个\*\*必须在早期解决的部署问题\*\*)

频域分支按公式 (4) 直接用 `torch.fft.fft2` 写是最自然的, 但:

```
torch.onnx.export(..., opset_version=17) -> UnsupportedOperatorError:
```

```
Exporting the operator 'aten::fft_fft2' to ONNX opset version 17 is not supported
```

**\*\*PyTorch 的 FFT 算子无法导出到 ONNX opset 17\*\*** (ONNX 有独立的 `DFT` 算子,

但 PyTorch 没有为 `fft\_fft2` 注册对应的符号函数)。

也就是说: 如果直接用 `torch.fft`, 你训练完的模型\*\*根本导不出 ONNX,

后面 OpenVINO 加速、桌面工具全部卡死\*\*。这个问题往往在项目后期才暴露, 届时已经来不及改结构。

解决办法: 把 2D-DFT 写成\*\*固定的矩阵乘法\*\*。

DFT 本质是线性变换:  $F = W_H \cdot x \cdot W_W^T$ , 其中

$$W_H[h, k] = \exp(-j \cdot 2\pi \cdot h \cdot k / H) / \sqrt{H}$$

$$W_W[v, l] = \exp(-j \cdot 2\pi \cdot v \cdot l / W) / \sqrt{W} \quad (\text{norm='ortho' 归一化})$$

对于实数输入  $x$ , 令  $A = W_H \cdot x$ ,  $B = W_W^T \cdot x$ , 则

$$\text{Re}(F) = A \cdot W_R^T \square B \cdot W_I^T$$

$$\text{Im}(F) = A \cdot W_I^T + B \cdot W_R^T$$

`fftshift` 也可以\*\*烘焙进矩阵的序\*\* (把行按  $(h + H/2) \% H$  排列),

于是整条链路只剩  $\text{matmul} / \text{sqrt} / \text{atan2}$  —— 全部是 ONNX opset 17 支持的算子。

**\*\*额外好处: CPU 上更快。\*\***  $224 \times 224$  的矩阵乘法一次约 11M MAC, 全流程 6 次约 67M MAC, 折算约 1~3 ms (Intel i7), 比 OpenVINO 里没有高效实现的 FFT 算子更可控。

两种模式:

dft\_mode="torch" : 训练用 (GPU 上 FFT 更快, 且数值最权威)

dft\_mode="matmul" : 导出/部署用 (数值与 torch 模式等价, 误差  $\sim 1e-5$ )

```
验证等价性: `python -m src.models.dft`
"""

```

```
from __future__ import annotations

```

```
import math

```

```
from typing import Tuple

```

```
import torch

```

```
import torch.nn as nn

```

```
# =====

```

```
def build_dft_matrices(n: int, shift: bool = True, ortho: bool = True
    ) -> Tuple[torch.Tensor, torch.Tensor]:

```

"""构造 1D DFT 的实部/虚部矩阵。

Args:

n: 变换长度

shift: 是否把 fftshift 烘焙进行序 (行索引  $r$  对应频率槽  $(r + n/2) \% n$ )

ortho: 是否使用  $1/\text{sqrt}(n)$  正交归一化 (与 `torch.fft.fft2(norm='ortho')` 一致)

```

Returns:
    (Wre, Wim), 均为 (n, n) 的 float32 张量
"""
k = torch.arange(n, dtype=torch.float64).view(1, -1)          # 输入索引
rows = torch.arange(n, dtype=torch.float64).view(-1, 1)      # 频率索引
if shift:
    rows = (rows + n // 2) % n
angle = -2.0 * math.pi * rows * k / n
scale = (1.0 / math.sqrt(n)) if ortho else 1.0
return (torch.cos(angle) * scale).float(), (torch.sin(angle) * scale).float()

# =====
class DFT2D(nn.Module):
    """2D-DFT + fftshift + 幅相分离 (公式 (4)(5)(6)), 支持 ONNX 导出。
    forward 输入 (B,1,H,W) 实数张量, 输出 (amp, phase) 均为 (B,1,H,W)。
    """
    def __init__(self, height: int = 224, width: int = 224, mode: str = "torch",
                  shift: bool = True, ortho: bool = True):
        super().__init__()
        assert mode in ("torch", "matmul"), f"未知 dft_mode: {mode}"
        self.mode = mode
        self.height, self.width = height, width
        self.shift, self.ortho = shift, ortho
        wre_h, wim_h = build_dft_matrices(height, shift, ortho)
        wre_w, wim_w = build_dft_matrices(width, shift, ortho)
        # persistent=False: 不写进 checkpoint (可随时重建, 省 1.6MB 权重体积)
        self.register_buffer("Wre_H", wre_h, persistent=False)
        self.register_buffer("Wim_H", wim_h, persistent=False)
        self.register_buffer("Wre_W", wre_w, persistent=False)
        self.register_buffer("Wim_W", wim_w, persistent=False)

    # =====
    def to_matmul(self) -> "DFT2D":
        """切换到可导出模式 (导出 ONNX 前必须调用)。"""
        self.mode = "matmul"
        return self

    def to_torch(self) -> "DFT2D":
        self.mode = "torch"
        return self

    # =====
    def _fft_shift_torch(self, spec: torch.Tensor) -> torch.Tensor:
        if not self.shift:
            return spec
        return torch.fft.fftshift(spec, dim=(-2, -1))

    def forward(self, x: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
        """x: (B,1,H,W) -> (amp (B,1,H,W), phase (B,1,H,W))"""
        if self.mode == "torch":
            spec = torch.fft.fft2(x, dim=(-2, -1), norm="ortho" if self.ortho else "backward")
            spec = self._fft_shift_torch(spec)
            real, imag = spec.real, spec.imag
        else:
            #  $F = Wre\_H \cdot x \cdot Wre\_W^T - Wim\_H \cdot x \cdot Wim\_W^T + j(Wre\_H \cdot x \cdot Wim\_W^T + Wim\_H \cdot x \cdot Wre\_W^T)$ 
            a = torch.matmul(self.Wre_H, x)          # (B,1,H,W)
            b = torch.matmul(self.Wim_H, x)

```

```

        real = torch.matmul(a, self.Wre_W.t()) - torch.matmul(b, self.Wim_W.t())
        imag = torch.matmul(a, self.Wim_W.t()) + torch.matmul(b, self.Wre_W.t())
        amp = torch.sqrt(real ** 2 + imag ** 2 + 1e-12)
        phase = torch.atan2(imag, real) # (-pi, pi]
        return amp, phase
# =====
def _self_test() -> None:
    """验证 matmul 模式与 torch FFT 模式的数值等价性。
    注意：相位比较要用**圆周差**`wrap(Δφ)`，因为 atan2 的值域  $(-\pi, \pi]$  有分支割线，
    在幅值趋近 0 的点上（相位本身无意义） $\Delta\phi$  会跳到  $\pm 2\pi$ 。
    """
    torch.manual_seed(0)
    ok = True
    for size in (64, 224):
        x = torch.randn(2, 1, size, size)
        d_t = DFT2D(size, size, mode="torch")
        d_m = DFT2D(size, size, mode="matmul")
        a1, p1 = d_t(x)
        a2, p2 = d_m(x)
        amp_err = float((a1 - a2).abs().max())
        d_phi = p1 - p2
        d_phi = torch.remainder(d_phi + math.pi, 2 * math.pi) - math.pi # 圆周差
        mask = a1 > 1e-2
        phase_err = float(d_phi[mask].abs().max()) if bool(mask.any()) else 0.0
        print(f" size={size:4d} 幅值最大误差={amp_err:.3e} "
              f"相位最大圆周误差={phase_err:.3e}")
        ok &= amp_err < 1e-4 and phase_err < 1e-3
    print("matmul 模式与 torch FFT 模式数值等价 ☐" if ok else "☐ 数值不等价")
    assert ok
if __name__ == "__main__":
    print("DFT2D 自检：")
    _self_test()
/* ===== 文件：src/models/cs_cam.py ===== */
"""1.1 (3) 通道-空间交叉注意力自适应特征融合模块（CS-CAM）
对应申报书公式 (13)~(17):
    F_spa' = Conv1x1(F_spa) in R^512 (13)
    F_freq' = Conv1x1(F_freq) in R^512 (14)
    F_spa'' = F_spa' * CA(F_spa') * SA(F_spa') (15)
    F_freq'' = F_freq' * CA(F_freq') * SA(F_freq') (16)
    F_fusion = F_spa'' + F_freq'' in R^512 (17)
其中 CA 为通道注意力（GAP + GMP -> 共享 MLP），SA 为空间注意力
（沿通道轴 GAP + GMP -> 拼接 -> 7x7 卷积 -> Sigmoid）。
-----
实现说明
-----
* 公式里 512 维写成向量，实际是 (B,512,14,14) 的特征图，1x1 卷积即通道映射。
* 通道注意力 MLP 的降维比 r=16 (512 -> 32 -> 512)，与 CBAM 一致。
* 空间注意力卷积核取 7x7 (CBAM 原文设定)，申报书未给核大小，此处按惯例取 7；
  若审阅要求严格，改 `spatial_kernel=3` 即可。
* 消融：`use_cross_attention=False` 时退化为 1x1 卷积后直接相加（无注意力加权）。
"""
from __future__ import annotations

```

```

import torch
import torch.nn as nn
# -----
class ChannelAttention(nn.Module):
    """通道注意力 CA(·): GAP/GMP -> 共享 MLP -> Sigmoid -> (B,C,1,1)。"""
    def __init__(self, channels: int, reduction: int = 16):
        super().__init__()
        hidden = max(4, channels // reduction)
        self.shared_mlp = nn.Sequential(
            nn.Conv2d(channels, hidden, 1, bias=False),
            nn.ReLU(inplace=True),
            nn.Conv2d(hidden, channels, 1, bias=False),
        )
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        avg = self.shared_mlp(x.mean(dim=(2, 3), keepdim=True))
        mx = self.shared_mlp(x.amax(dim=(2, 3), keepdim=True))
        return torch.sigmoid(avg + mx)

class SpatialAttention(nn.Module):
    """空间注意力 SA(·): 沿通道轴 GAP/GMP -> 拼接 -> 卷积 -> Sigmoid -> (B,1,H,W)。"""
    def __init__(self, kernel: int = 7):
        super().__init__()
        pad = kernel // 2
        self.conv = nn.Conv2d(2, 1, kernel, 1, pad, bias=False)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        avg = x.mean(dim=1, keepdim=True)
        mx = x.amax(dim=1, keepdim=True)
        return torch.sigmoid(self.conv(torch.cat([avg, mx], dim=1)))

# -----
class CSCAM(nn.Module):
    """通道-空间交叉注意力融合模块。"""
    def __init__(self, spa_dim: int = 896, freq_dim: int = 256, dim: int = 512,
                 use_cross_attention: bool = True, reduction: int = 16, spatial_kernel: int = 7):
        super().__init__()
        self.use_cross_attention = use_cross_attention
        self.proj_spa = nn.Sequential(
            nn.Conv2d(spa_dim, dim, 1, bias=False), nn.BatchNorm2d(dim), nn.ReLU(inplace=True)
        )
        self.proj_freq = nn.Sequential(
            nn.Conv2d(freq_dim, dim, 1, bias=False), nn.BatchNorm2d(dim), nn.ReLU(inplace=True)
        )
        if use_cross_attention:
            self.ca_spa, self.sa_spa = ChannelAttention(dim, reduction), SpatialAttention(spatial_kernel)
            self.ca_freq, self.sa_freq = ChannelAttention(dim, reduction), SpatialAttention(spatial_kernel)
        self.dim = dim
    def forward(self, f_spa: torch.Tensor, f_freq: torch.Tensor) -> torch.Tensor:
        """入参为特征图 (B,896,14,14) / (B,256,14,14)，返回 (B,512,14,14)。"""
        p_spa = self.proj_spa(f_spa) # 公式 (13)
        p_freq = self.proj_freq(f_freq) # 公式 (14)
        if self.use_cross_attention:
            a_spa = p_spa * self.ca_spa(p_spa) * self.sa_spa(p_spa) # 公式 (15)
            a_freq = p_freq * self.ca_freq(p_freq) * self.sa_freq(p_freq) # 公式 (16)
        else:

```

```

        a_spa, a_freq = p_spa, p_freq
    return a_spa + a_freq          # 公式 (17)  F_fusion
@torch.no_grad()
def sanity_check(self) -> dict:
    self.eval()
    f_spa = torch.randn(2, 896, 14, 14)
    f_freq = torch.randn(2, 256, 14, 14)
    out = self.forward(f_spa, f_freq)
    return {"F_spa": tuple(f_spa.shape), "F_freq": tuple(f_freq.shape),
            "F_fusion": tuple(out.shape), "期望": "(2,512,14,14)"}

/* ===== 文件: src/models/vib.py ===== */
"""1.2 分层 VIB 信息瓶颈特征提纯模块
对应申报书公式 (18)~(26):
    L_VIB = -E_q(z|x)[log q(y|z)] + beta * KL(q(z|x) || r(z))          (19)
    mu     = MLP_mu(F_fusion)          in R^256                        (20)
    sigma  = Softplus(MLP_sigma(F_fusion)) in R^256                    (21)
    Softplus(x) = log(1 + e^x)          (22)
    z       = mu + eps * sigma,  eps ~ N(0,1)    (重参数技巧)          (23)
    beta(t) = 0, t < 20
             = 0.1*(t-20)/20, 20 <= t < 40      (24)
             = 0.1, t >= 40
    KL = clip(KL(q(z|x) || N(0,1)), 0, 10)      (25)
三重稳定训练策略: beta 退火 / KL 散度裁剪 / 梯度裁剪(L2<=5.0, 见 engine/trainer)。
"分层" 的含义: VIB 只挂在**分类支路的最后一层**, 不作用于底层共享特征与定位支路,
从根本上避免压缩细粒度空间信息。
"""

from __future__ import annotations
import math
from typing import Tuple
import torch
import torch.nn as nn
# -----
def beta_schedule(epoch: int, beta_max: float = 0.1,
                  warm_start: int = 20, warm_end: int = 40) -> float:
    """公式 (24) 的分段 beta 退火 (此处实为"分段升温")。"""
    if epoch < warm_start:
        return 0.0
    if epoch < warm_end:
        return beta_max * (epoch - warm_start) / max(1, (warm_end - warm_start))
    return beta_max
# -----
class HierarchicalVIB(nn.Module):
    """分层变分信息瓶颈: 512 -> 256 -> (mu, log_sigma), 重参数采样得 z ∈ R^256。
    返回 (z, kl, mu, sigma), KL 已按公式 (25) 裁剪到 [kl_min, kl_max]。
    -----
    工程要点 1: σ 的初始化
    -----
    KL(N(μ, σ^2) || N(0,1)) = 0.5 * Σ (μ^2 + σ^2 log σ^2)
    若把 σ 初始化得很小 (例如 0.1), log σ^2 项会让 KL 起步就有几百,
    一上来就被裁到上限, 反向梯度直接被 clamp 吃掉 —— 表现为"VIB 完全不工作"。
    正确做法: σ 初值取 1 (此时 μ=0、σ=1 时 KL=0, 是 KL 的全局最小值),
    再让模型自己学。Softplus(b) = 1 => b = log(e^1 - 1) ≈ 0.5413。
    """

```

---

工程要点 2: KL 裁剪不能杀掉梯度（与公式 25 的重要澄清）

---

`torch.clamp` 在区间外梯度为 0。若 KL 常态性地  $> 10$ ，公式 (25) 就会把 VIB 正则项变成“常数”，等于\*\*静默关闭信息瓶颈\*\*，而且论文里完全看不出来。

本实现提供两种口径，默认选更稳妥的组合：

- \* `kl_reduction = "mean"`（对 batch 和 latent 维度都取均值）
  - 数值落在  $0.0x \sim \text{几}$  的量级，此时  $[0, 10]$  的裁剪是“安全阀”而非“常态”，与公式 (25) 的字面语义一致；
- \* `kl_reduction = "sum"`（对 latent 维求和、对 batch 取均值）
  - 即 Alemi 原始 VIB 写法，数值常在几十到几百， $[0, 10]$  会常态触发；
- \* `kl_clip_grad_through = True`（默认）：前向仍是硬裁剪（数值满足公式 25），反向按恒等传递，保证 KL 始终有梯度、VIB 不会静默失效。

"""

```
def __init__(self, in_dim: int = 512, hidden_dim: int = 256, latent_dim: int = 256,
              kl_clip_min: float = 0.0, kl_clip_max: float = 10.0,
              kl_reduction: str = "mean", kl_clip_grad_through: bool = True):
    super().__init__()
    # 第一层: 512 -> 256 (公式 (20) (21) 共用的第一层 MLP)
    self.fc_shared = nn.Sequential(
        nn.Linear(in_dim, hidden_dim),
        nn.LayerNorm(hidden_dim),
        nn.ReLU(inplace=True),
    )
    # 第二层分别输出 mu 与 sigma (sigma 用 Softplus 保证非负)
    self.fc_mu = nn.Linear(hidden_dim, latent_dim)
    self.fc_sigma = nn.Linear(hidden_dim, latent_dim)
    self.softplus = nn.Softplus()
    self.kl_clip_min = kl_clip_min
    self.kl_clip_max = kl_clip_max
    self.kl_reduction = kl_reduction
    self.kl_clip_grad_through = kl_clip_grad_through
    self.latent_dim = latent_dim
    # 初始化:  $\mu$  初值为 0;  $\sigma$  初值为 1 (Softplus(b)=1  $\Rightarrow$   $b=\log(e-1)\approx 0.5413$ ),
    # 这样 KL 从 0 起步、稳步增长, 避免"一上来就被裁到上限"。
    nn.init.zeros_(self.fc_mu.weight)
    nn.init.zeros_(self.fc_mu.bias)
    nn.init.normal_(self.fc_sigma.weight, std=0.01)
    nn.init.constant_(self.fc_sigma.bias, math.log(math.e - 1.0))
# -----
def encode(self, f_fusion: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """F_fusion 可以是 (B,512)、也可以是 (B,512,14,14) (内部自动 GAP)。"""
    if f_fusion.dim() == 4:
        f_fusion = f_fusion.mean(dim=(2, 3))
    h = self.fc_shared(f_fusion)
    mu = self.fc_mu(h)  # 公式 (20)
    sigma = self.softplus(self.fc_sigma(h)) + 1e-4  # 公式 (21) (22)
    return mu, sigma
def reparameterize(self, mu: torch.Tensor, sigma: torch.Tensor,
                   sample: bool = True) -> torch.Tensor:
    """公式 (23) 重参数技巧; eval 阶段可用 sample=False 取 mu (确定性推理)。"""
    if not sample:
```

```

        return mu
    eps = torch.randn_like(sigma)
    return mu + eps * sigma
def kl_divergence(self, mu: torch.Tensor, sigma: torch.Tensor) -> torch.Tensor:
    """KL(N(mu, sigma^2) || N(0,1))。
    kl_reduction="mean" -> 对 batch 与 latent 维都取均值（默认，配合 [0,10] 裁剪）
    kl_reduction="sum" -> 对 latent 维求和、对 batch 取均值（Alemi 原始写法）
    """
    kl = 0.5 * (mu.pow(2) + sigma.pow(2) - torch.log(sigma.pow(2) + 1e-8) - 1.0)
    if self.kl_reduction == "sum":
        return kl.sum(dim=-1).mean()
    return kl.mean()
def _clip_kl(self, kl: torch.Tensor) -> torch.Tensor:
    """公式 (25) KL 裁剪 [0,10]。
    数值上做硬裁剪（满足公式语义）；`kl_clip_grad_through=True` 时反向
    按恒等传递，避免 KL 常态化超上限后 VIB 静默失效。
    """
    clipped = torch.clamp(kl, self.kl_clip_min, self.kl_clip_max)
    if self.kl_clip_grad_through:
        clipped = kl + (clipped - kl).detach()
    return clipped
def forward(self, f_fusion: torch.Tensor, sample: bool = True,
            beta: float = 0.0) -> dict:
    mu, sigma = self.encode(f_fusion)
    z = self.reparameterize(mu, sigma, sample)
    kl_raw = self.kl_divergence(mu, sigma)
    kl = self._clip_kl(kl_raw) # 公式 (25)
    return {"z": z, "mu": mu, "sigma": sigma, "kl_raw": kl_raw, "kl": kl,
            "beta": beta, "kl_term": beta * kl}
# -----
@torch.no_grad()
def sanity_check(self) -> dict:
    self.eval()
    x = torch.randn(4, 512, 14, 14)
    out = self.forward(x, sample=True, beta=0.1)
    return {
        "mu": tuple(out["mu"].shape),
        "sigma": tuple(out["sigma"].shape),
        "z": tuple(out["z"].shape),
        "sigma_min/max": (float(out["sigma"].min()), float(out["sigma"].max())),
        "kl_raw": float(out["kl_raw"]),
        "kl_clipped": float(out["kl"]),
        "beta(10)/(30)/(50)": (
            beta_schedule(10), beta_schedule(30), beta_schedule(50)
        ),
    }

```

/\* ===== 文件: src/models/gradient\_stop.py ===== \*/

"""1.3 (1) 梯度隔离机制 —— 梯度停止层 (Gradient Stop)

对应申报书 1.3(1)：在前向传播时恒等传递，在反向传播时阻断梯度回传。

在 F\_fusion 与分类支路之间放 GS1，在 F\_fusion 与定位支路之间放 GS2。

□ 实现说明：这里必须讲清楚，否则论文里会被审稿人追问

申报书列出的三条性质：

- ① 分类支路的梯度仅回传至 VIB 模块和 F\_fusion
- ② 定位支路的梯度仅回传至定位头和 F\_fusion
- ③ F\_fusion 同时接收两条支路的梯度

在"共享 F\_fusion + 两个兄弟头"这种朴素结构下，① ② ③ \*\*是自动成立的\*\*：

分类头与定位头之间本来就没有计算图连线，梯度不可能跨支路串扰。

也就是说，若把 GS 实现为简单恒等映射，它在数学上是"空操作"，不会给模型带来任何额外收益——这点在写论文/答辩时必须主动说明，否则"加了梯度停止层"这句话站不住脚。

因此本模块提供两种可配置语义，把"梯度隔离"变成\*\*真实有效\*\*的机制：

- A) `stop=False`（默认，对应申报书描述的前向恒等行为）  
恒等映射，用于显式标注计算图中的解耦点，工程上便于可视化与调试。

- B) `stop=True`（严格解耦 / 真正的梯度隔离）

阻断该支路梯度回传到共享空频融合特征 F\_fusion。语义为：

- 打开 cls 侧 GS → 定位任务的梯度不再塑造共享特征，共享特征完全由分类任务驱动（分类优先，泛化性优先）；
- 打开 loc 侧 GS → 分类任务的梯度不再塑造共享特征，共享特征完全由像素级定位任务驱动（定位细粒度优先）。

这等价于"任务独立的特征子空间优化"，能显著缓解多任务梯度冲突

（与 PCGrad / GradNorm 属于同一类思路，但零额外计算开销），

是消融实验中可以真正观察到指标差异的开关。

推荐实验配置：

- \* 三阶段训练中，阶段 1/2 由于另一支路被冻结，GS 无影响；
- \* 阶段 3 联合微调时，`gs\_stop\_cls=True` 常用于"定位精度优先"的场景，而 `gs\_stop\_loc=True` 常用于"跨模型泛化优先"的场景。

"""

from \_\_future\_\_ import annotations

import torch

import torch.nn as nn

# -----

class \_GradientStopFunction(torch.autograd.Function):

"""前向恒等，反向按需清零。"""

@staticmethod

def forward(ctx, x: torch.Tensor, stop: bool):

ctx.stop = bool(stop)

return x.view\_as(x)

@staticmethod

def backward(ctx, grad\_output: torch.Tensor):

if ctx.stop:

return torch.zeros\_like(grad\_output), None

return grad\_output, None

def grad\_stop(x: torch.Tensor, stop: bool = True) -> torch.Tensor:

"""函数式梯度停止层。stop=True 时阻断梯度回传。"""

if x.requires\_grad:

return \_GradientStopFunction.apply(x, stop)

return x

# -----

class GradientStopLayer(nn.Module):

"""梯度停止层 (nn.Module 形式，便于在 `nn.ModuleDict` 中按阶段开关)。

Args:

stop: True → 反向阻断（严格解耦）；False → 恒等（默认，符合申报书描述）

```

        name: 用于日志/调试图标注 (如 "GS1_cls" / "GS2_loc")
使用示例::
    f_fusion = fusion(f_spa, f_freq)          # 共享特征
    z = vib(grad_stop(f_fusion, gs1.stop))    # 分类支路
    mask = loc_head(grad_stop(f_fusion, gs2.stop)) # 定位支路
"""

def __init__(self, stop: bool = False, name: str = "GS"):
    super().__init__()
    self.stop = stop
    self.name = name

def forward(self, x: torch.Tensor) -> torch.Tensor:
    return grad_stop(x, self.stop)

def extra_repr(self) -> str:
    return f"name={self.name}, stop={self.stop}"

# -----
@staticmethod
def verify(stop: bool = True) -> dict:
    """自检：验证梯度阻断是否生效。"""
    x = torch.ones(3, 4, requires_grad=True)
    y = grad_stop(x, stop).sum()
    y.backward()
    blocked = bool(torch.allclose(x.grad, torch.zeros_like(x.grad)))
    return {"stop": stop, "上游梯度全零": blocked,
            "grad_shape": tuple(x.grad.shape)}

/* ===== 文件: src/models/mobile_unetv2.py ===== */
"""1.3 (3) 定位支路 —— Mobile-UNetv2 定位头 + Canny 边缘监督分支
对应申报书 1.3(3) 与公式 (28) (29):
    M_pred = sigmoid(Mobile-UNetv2(F_fusion)) in [0,1]^(224x224)          (28)
    L_edge = BCE(M_edge_pred, M_edge_gt), M_edge_gt = Canny(M_gt)        (29)
Mobile-UNetv2 结构 (申报书原文):
    编码器: 4 个倒残差块, 步长 1/2/2/2, 通道 64 -> 128 -> 256 -> 512, ReLU6
    解码器: 3 个上采样块 (双线性 2x 上采样 -> 与编码器对应层拼接 -> 3x3 卷积融合)
    输出层: 1x1 卷积 -> 1 通道 -> Sigmoid -> 224x224x1
输入: F_fusion (B, 512, 14, 14) (**未经 VIB 压缩**, 避免细粒度信息丢失)
尺寸流: (14,14) -> 14 -> 7 -> 4 -> 2 -> 上采样 4 -> 7 -> 14 -> 224
(解码器上采样使用 `interpolate(size=skip.shape)` 对齐, 规避奇数尺寸误差)
参数量约 3M 量级, 满足申报书"轻量、适合端侧"的要求。
"""

from __future__ import annotations
from typing import List, Tuple
import torch
import torch.nn as nn
import torch.nn.functional as F
# -----
class InvertedResidual(nn.Module):
    """MobileNetV2 倒残差块: 1x1 逐点(升维) -> 3x3 深度卷积 -> 1x1 逐点(降维), ReLU6."""
    def __init__(self, in_ch: int, out_ch: int, stride: int = 1, expand_ratio: float = 2.0):
        super().__init__()
        hidden = int(round(in_ch * expand_ratio))
        hidden = max(hidden, out_ch)
        self.use_skip = (stride == 1 and in_ch == out_ch)
        layers: List[nn.Module] = []

```

```

        if hidden != in_ch:
            layers += [nn.Conv2d(in_ch, hidden, 1, 1, 0, bias=False),
                       nn.BatchNorm2d(hidden), nn.ReLU6(inplace=True)]
        layers += [
            nn.Conv2d(hidden, hidden, 3, stride, 1, groups=hidden, bias=False),
            nn.BatchNorm2d(hidden), nn.ReLU6(inplace=True),
            nn.Conv2d(hidden, out_ch, 1, 1, 0, bias=False),
            nn.BatchNorm2d(out_ch),
        ]
        self.conv = nn.Sequential(*layers)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        out = self.conv(x)
        return x + out if self.use_skip else out
class UpBlock(nn.Module):
    """上采样块: 双线性插值 2x -> 与编码器特征拼接 -> 3x3 卷积融合。"""
    def __init__(self, in_ch: int, skip_ch: int, out_ch: int):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch + skip_ch, out_ch, 3, 1, 1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )
    def forward(self, x: torch.Tensor, skip: torch.Tensor) -> torch.Tensor:
        x = F.interpolate(x, size=skip.shape[-2:], mode="bilinear", align_corners=False)
        return self.conv(torch.cat([x, skip], dim=1))
# -----
class MobileUNetV2(nn.Module):
    def __init__(self, in_dim: int = 512, out_size: int = 224,
                 encoder_channels: Tuple[int, ...] = (64, 128, 256, 512),
                 encoder_strides: Tuple[int, ...] = (1, 2, 2, 2),
                 decoder_channels: Tuple[int, ...] = (256, 128, 64),
                 refine_channels: int = 32):
        super().__init__()
        self.out_size = out_size
        ec = list(encoder_channels)
        # ---- 编码器: 4 个倒残差块
        self.enc1 = InvertedResidual(in_dim, ec[0], encoder_strides[0], expand_ratio=1.0)
        self.enc2 = InvertedResidual(ec[0], ec[1], encoder_strides[1], expand_ratio=2.0)
        self.enc3 = InvertedResidual(ec[1], ec[2], encoder_strides[2], expand_ratio=2.0)
        self.enc4 = InvertedResidual(ec[2], ec[3], encoder_strides[3], expand_ratio=2.0)
        # ---- 解码器: 3 个上采样块 + 与编码器对应层拼接
        dc = list(decoder_channels)
        self.dec3 = UpBlock(ec[3], ec[2], dc[0]) # 2x2 -> 4x4
        self.dec2 = UpBlock(dc[0], ec[1], dc[1]) # -> 7x7
        self.dec1 = UpBlock(dc[1], ec[0], dc[2]) # -> 14x14
        # ---- 输出层: 上采样到 224 -> 3x3 卷积 -> 1x1 卷积 -> 1 通道
        self.refine_channels = refine_channels
        self.refine = nn.Sequential(
            nn.Conv2d(dc[2], refine_channels, 3, 1, 1, bias=False),

```

```

        nn.BatchNorm2d(refine_channels),
        nn.ReLU(inplace=True),
    )
    self.out_conv = nn.Conv2d(refine_channels, 1, 1)
    # 供边缘监督头复用 (= refine 的输出通道数)
    self.feature_dim = refine_channels
def forward(self, f_fusion: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """返回 (mask_logits (B,1,224,224), 解码器特征 (B,64,224,224))。"""
    e1 = self.enc1(f_fusion)      # (B, 64, 14, 14)
    e2 = self.enc2(e1)            # (B,128, 7, 7)
    e3 = self.enc3(e2)            # (B,256, 4, 4)
    e4 = self.enc4(e3)            # (B,512, 2, 2)
    d3 = self.dec3(e4, e3)        # (B,256, 4, 4)
    d2 = self.dec2(d3, e2)        # (B,128, 7, 7)
    d1 = self.dec1(d2, e1)        # (B, 64, 14, 14)
    feat = F.interpolate(d1, size=(self.out_size, self.out_size),
                        mode="bilinear", align_corners=False)
    feat = self.refine(feat)      # (B, 32, 224, 224)
    logits = self.out_conv(feat) # (B, 1, 224, 224) 公式 (28) 的 logits
    return logits, feat
class EdgeHead(nn.Module):
    """边缘监督分支：输入定位支路解码特征，1x1 卷积输出 224x224x1 边缘预测（公式 29）。"""
    def __init__(self, in_dim: int = 32, mid: int = 16):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_dim, mid, 3, 1, 1, bias=False),
            nn.BatchNorm2d(mid),
            nn.ReLU(inplace=True),
            nn.Conv2d(mid, 1, 1),
        )
    def forward(self, feat: torch.Tensor) -> torch.Tensor:
        return self.conv(feat)
    @staticmethod
    def canny_edge(mask: torch.Tensor, low: int = 50, high: int = 150) -> torch.Tensor:
        """从 GT 掩码提取边缘 M_edge_gt（申报书：Canny 低阈值 50 / 高阈值 150）。
        纯 PyTorch 实现（Sobel 梯度 + 双阈值 + 非极大值抑制的简化版），
        避免训练时引入 OpenCV 依赖与 CPU-GPU 同步开销。
        若数据管线已离线生成边缘图，直接用离线结果更好。
        """
        if mask.dim() == 3:
            mask = mask.unsqueeze(1)
        m = mask.float()
        kx = torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]],
                        device=m.device, dtype=m.dtype).view(1, 1, 3, 3)
        ky = kx.transpose(-1, -2)
        pad = F.pad(m, (1, 1, 1, 1), mode="replicate")
        gx = F.conv2d(pad, kx)
        gy = F.conv2d(pad, ky)
        # 注意：不要把 eps 加在 sqrt 里面。
        # 若写成 sqrt(gx**2+gy**2+1e-8)，全零掩码会得到 mag==1e-4、amax=1e-4，
        # 归一化后 mag==1.0，于是"整幅图都是边缘"——这是极易踩的坑。
        mag = torch.sqrt(gx**2 + gy**2)

```

```

        scale = mag.amax(dim=(2, 3), keepdim=True).clamp_min(1e-6)
        mag = mag / scale
        lo, hi = low / 255.0, high / 255.0
        strong = (mag >= hi).float()
        weak = ((mag >= lo) & (mag < hi)).float()
        # 弱边缘沿 3x3 邻域找到强边缘则保留 (滞后阈值)
        dilated_strong = F.max_pool2d(strong, 3, 1, 1)
        edge = torch.clamp(strong + weak * dilated_strong, 0, 1)
        return edge

# -----
class LocalizationBranch(nn.Module):
    """定位支路整体: MobileUNetV2 定位头 + 可选边缘监督头。"""
    def __init__(self, cfg: dict):
        super().__init__()
        loc = cfg["model"]["localization"]
        self.head = MobileUNetV2(
            in_dim=cfg["model"]["fusion"]["dim"],
            out_size=cfg["data"]["image_size"],
            encoder_channels=tuple(loc.get("encoder_channels", [64, 128, 256, 512])),
            encoder_strides=tuple(loc.get("encoder_strides", [1, 2, 2, 2])),
        )
        self.use_edge_head = loc.get("use_edge_head", True)
        self.edge_head = EdgeHead(self.head.feature_dim) if self.use_edge_head else None
    def forward(self, f_fusion: torch.Tensor) -> dict:
        mask_logits, feat = self.head(f_fusion)
        out = {
            "mask_logits": mask_logits,
            "mask_prob": torch.sigmoid(mask_logits),
            "loc_feat": feat,
        }
        if self.edge_head is not None:
            edge_logits = self.edge_head(feat)
            out["edge_logits"] = edge_logits
            out["edge_prob"] = torch.sigmoid(edge_logits)
        return out
    def num_millions(self) -> float:
        n = sum(p.numel() for p in self.head.parameters())
        if self.edge_head is not None:
            n += sum(p.numel() for p in self.edge_head.parameters())
        return n / 1e6

/* ===== 文件: src/losses/multi_task.py ===== */
"""1.4 不确定性加权多阶段联合损失函数
对应申报书公式 (26) (29) (30) (31):

$$L_{VIB} = -(1/N) * \sum_i \log q(y_i | z_i) + \beta(t) * KL \quad (26)$$

其中 KL 已按公式 (25) 裁剪到 [0,10]

$$L_{edge} = BCE(M_{edge\_pred}, M_{edge\_gt}) \quad (29)$$


$$L' = (L - \mu_L) / \sigma_L \quad \# Z\text{-score 归一化, 滑动窗口 100 批次} \quad (30)$$


$$L_{total} = 1/(2*\sigma_1^2) * L'_{VIB} + 1/(2*\sigma_2^2) * L'_{BCE} \\ + 1/(2*\sigma_3^2) * L'_{Dice} + 1/(2*\sigma_4^2) * L'_{edge} \\ + \log(\sigma_1*\sigma_2*\sigma_3*\sigma_4) \quad (31)$$


```

实现细节说明

---

```

1. 数值稳定的参数化: 令  $s_i = \log(\sigma_i^2)$ , 则
     $1/(2 \sigma_i^2) = 0.5 * \exp(-s_i)$ 
     $\log(\sigma_1^2 \sigma_2^2 \sigma_3^2 \sigma_4^2) = 0.5 * \sum_i s_i$ 
    于是  $L_{total} = \sum_i [0.5 * \exp(-s_i) * L'_i + 0.5 * s_i]$ , 全程无除法、
    无开方, 且梯度有界 (这是 Kendall 2018 的原始写法)。
2. Z-score 归一化用"滑动窗口统计量", 且**统计量 detach** (只让分子回传梯度)。
    否则归一化的分母也会被优化, 导致损失尺度失控。
    窗口默认 100 个批次, 用 deque 实现, 同时用 EMA 兜底 (窗口未满时)。
3. 若 `uncertainty_weighting=False`, 退化为固定权重版 (消融基线)。
4. `terms` 控制当前阶段参与计算的损失项 (三阶段训练按阶段启用)。
"""

from __future__ import annotations
from collections import deque
from typing import Dict, Iterable, List, Optional
import torch
import torch.nn as nn
import torch.nn.functional as F
from ..models.mobile_unetv2 import EdgeHead
# =====
# 基础损失项
# =====

def vib_loss(cls_logits: torch.Tensor, labels: torch.Tensor,
             kl: torch.Tensor, beta: float) -> torch.Tensor:
    """公式 (26): 交叉熵 ( $= -\log q(y|z)$ ) +  $\beta * KL$ ."""
    ce = F.cross_entropy(cls_logits, labels)
    return ce + beta * kl

def bce_loss(mask_logits: torch.Tensor, mask_gt: torch.Tensor,
             valid: Optional[torch.Tensor] = None) -> torch.Tensor:
    """逐像素 BCE。`valid` (B,) 为 bool 时, 只对含有效掩码的样本求平均。
    混合批次 (ForenSynths 无掩码 + CASIA 有掩码) 必须用 valid 过滤,
    否则补零掩码会把定位支路教成"全部预测为真实"。
    """
    per_sample = F.binary_cross_entropy_with_logits(
        mask_logits, mask_gt, reduction="none"
    ).flatten(1).mean(dim=1)
    if valid is not None:
        if valid.sum() == 0:
            return mask_logits.sum() * 0.0
        return per_sample[valid].mean()
    return per_sample.mean()

def dice_loss(mask_logits: torch.Tensor, mask_gt: torch.Tensor, smooth: float = 1.0,
             valid: Optional[torch.Tensor] = None) -> torch.Tensor:
    """1 - Dice 系数 (逐样本计算后取均值, 避免大目标主导)。"""
    prob = torch.sigmoid(mask_logits)
    b = prob.shape[0]
    p = prob.reshape(b, -1)
    g = mask_gt.reshape(b, -1)
    inter = (p * g).sum(dim=1)
    denom = p.sum(dim=1) + g.sum(dim=1)
    dice = (2 * inter + smooth) / (denom + smooth)
    per_sample = 1.0 - dice

```

---

```

    if valid is not None:
        if valid.sum() == 0:
            return mask_logits.sum() * 0.0
        return per_sample[valid].mean()
    return per_sample.mean()

def edge_loss(edge_logits: torch.Tensor, edge_gt: torch.Tensor,
              valid: Optional[torch.Tensor] = None) -> torch.Tensor:
    """公式 (29): 边缘二元交叉熵。"""
    per_sample = F.binary_cross_entropy_with_logits(
        edge_logits, edge_gt, reduction="none"
    ).flatten(1).mean(dim=1)
    if valid is not None:
        if valid.sum() == 0:
            return edge_logits.sum() * 0.0
        return per_sample[valid].mean()
    return per_sample.mean()

# =====
# 公式 (30) Z-score 归一化
# =====

class ZScoreNormalizer:
    """按损失项维护滑动窗口统计量, 实现  $L' = (L - \mu_L) / \sigma_L$ 。
    注意: 窗口未满时用 Welford 在线均值/方差兜底, 保证第一步就能归一化。
    """

    def __init__(self, terms: Iterable[str], window: int = 100, eps: float = 1e-6):
        self.terms = list(terms)
        self.window = max(2, window)
        self.eps = eps
        self.history: Dict[str, deque] = {t: deque(maxlen=self.window) for t in self.terms}
        self._ema_mean: Dict[str, float] = {t: None for t in self.terms}
        self._ema_var: Dict[str, float] = {t: None for t in self.terms}
        self._count: int = 0

    @torch.no_grad()
    def _update(self, name: str, value: float) -> None:
        self.history[name].append(value)
        # EMA 兜底 (alpha=0.05), 窗口满后 EMA 与窗口统计量混合
        if self._ema_mean[name] is None:
            self._ema_mean[name] = value
            self._ema_var[name] = 0.0
        else:
            a = 0.05
            diff = value - self._ema_mean[name]
            self._ema_mean[name] += a * diff
            self._ema_var[name] = (1 - a) * (self._ema_var[name] + a * diff * diff)

    def stats(self, name: str) -> tuple:
        """返回 (mu, sigma)。
        某个损失项**从未在当前阶段被更新过**时 (例如阶段一只训练分类,
        bce/dice/edge 的统计量为空), 退化为恒等归一化 (0, 1),
        而不是抛异常 —— 否则三阶段训练在第一次验证时就会崩。
        """
        hist = self.history.get(name)
        if hist and len(hist) >= max(4, self.window // 4):
            t = torch.tensor(list(hist), dtype=torch.float32)

```

```

        return float(t.mean()), float(t.std(unbiased=False).clamp_min(self.eps))
    if self._ema_mean.get(name) is None:
        return 0.0, 1.0
    return float(self._ema_mean[name]), float(max(self._ema_var[name] ** 0.5, self.eps))
def normalize(self, name: str, value: torch.Tensor, update: bool = True) -> torch.Tensor:
    """返回归一化后的损失（保留 value 的梯度，统计量分离）。"""
    if update and self.training_enabled:
        self._update(name, float(value.detach()))
    mu, sigma = self.stats(name)
    return (value - mu) / sigma
training_enabled: bool = True
def state_dict(self) -> dict:
    return {f"hist_{k}": list(v) for k, v in self.history.items()}
def load_state_dict(self, sd: dict) -> None:
    for k, v in sd.items():
        if k.startswith("hist_"):
            name = k[len("hist_"):]
            if name in self.history:
                self.history[name] = deque(v, maxlen=self.window)
# =====
# 公式 (31) 同方差不确定性加权
# =====
class UncertaintyWeighting(nn.Module):
    """可学习不确定性参数。s_i = log(sigma_i^2) 初始化为 0（即 sigma=1，等权起步）。
    □ 为什么必须钳位 s_i
    -----
    L_total = Σ [ 0.5*exp(-s_i)*L'_i + 0.5*s_i ], 把 s_i 看成自变量求导：
        dL/ds_i = -0.5*exp(-s_i)*L'_i + 0.5 = 0 => s_i* = ln(L'_i)
    **只有当 L'_i > 0 时才有极小值**。而公式 (30) 的 Z-score 归一化把每项损失
    中心化到 0 附近，约一半批次会出现 L'_i < 0；此时 dL/ds_i 恒为正，
    即 L 关于 s_i 单调递增 → 最优解是 s_i → -∞，损失无下界，
    训练会表现为「日志里的 total loss 一路向负无穷漂、权重发散」。
    这就是申报书公式 (30) 与 (31) 直接串联时的数学不一致之处。
    这里给 s_i 加一个对称区间钳位作为工程保护：越界后 clamp 的梯度为 0，
    参数自动停住，训练不会发散。默认 [-4, 4] 对应 sigma ∈ [0.135, 7.39]，
    权重 0.5*exp(±4) ∈ [0.009, 27.3]，覆盖了正常需要的动态范围。
    若要彻底消除不一致，应把加权作用在**未归一化的非负原始损失**上，
    只把 Z-score 用于日志展示（见 docs/08）。
    """
    def __init__(self, terms: List[str], s_min: float = -4.0, s_max: float = 4.0):
        super().__init__()
        self.terms = list(terms)
        self.log_var = nn.Parameter(torch.zeros(len(self.terms)))
        self.s_min, self.s_max = float(s_min), float(s_max)
        self._clamped = False
    def forward(self, losses: Dict[str, torch.Tensor]) -> Dict[str, torch.Tensor]:
        total = 0.0
        detail = {}
        for i, t in enumerate(self.terms):
            if t not in losses:
                continue
            s_raw = self.log_var[i]

```

```

        s = torch.clamp(s_raw, self.s_min, self.s_max)
        if not self._clamped and float(s_raw.detach()) != float(s.detach()):
            self._clamped = True      # 只提示一次，避免刷屏
        w = 0.5 * torch.exp(-s)
        reg = 0.5 * s
        term = w * losses[t] + reg
        total = total + term
        detail[f"w_{t}"] = float(w.detach())
        detail[f"sigma_{t}"] = float(torch.exp(0.5 * s).detach())
        return {"total": total, "detail": detail, "clamped": self._clamped}
    @torch.no_grad()
    def weights(self) -> Dict[str, float]:
        return {t: float(0.5 * torch.exp(-torch.clamp(self.log_var[i],
                                                         self.s_min, self.s_max)))
                for i, t in enumerate(self.terms)}
# =====
# 组合损失
# =====
class MultiTaskLoss(nn.Module):
    """多任务联合损失：归一化 -> 不确定性加权（或固定权重）。"""
    def __init__(self, cfg: dict):
        super().__init__()
        lcfg = cfg["loss"]
        self.terms = list(lcfg.get("terms", ["vib", "bce", "dice", "edge"]))
        self.norm_mode = lcfg.get("normalization", "zscore")
        self.normalizer = ZScoreNormalizer(self.terms, lcfg.get("zscore_window", 100))
        self.use_uncertainty = lcfg.get("uncertainty_weighting", True)
        self.fixed_weights = lcfg.get("weights", {})
        self.dice_smooth = lcfg.get("dice_smooth", 1.0)
        if self.use_uncertainty:
            self.weighting = UncertaintyWeighting(
                self.terms,
                s_min=float(lcfg.get("uncertainty_s_min", -4.0)),
                s_max=float(lcfg.get("uncertainty_s_max", 4.0)))
        else:
            self.weighting = None
# -----
    def forward(
        self,
        outputs: Dict[str, torch.Tensor],
        labels: torch.Tensor,
        mask_gt: Optional[torch.Tensor],
        beta: float = 0.0,
        active_tasks: Optional[List[str]] = None,
        edge_gt: Optional[torch.Tensor] = None,
        update_norm: bool = True,
        mask_valid: Optional[torch.Tensor] = None,
    ) -> Dict[str, object]:
        """计算总损失。
        Args:
            outputs: 模型前向输出
            labels: (B,) 0=真实 1=伪造

```

```

        if c == 2:
            item.setForeground(QColor("#c0392b" if res.is_fake else "#1e8e3e"))
        self.table.setItem(r, c, item)
def _on_row_changed(self) -> None:
    rows = {i.row() for i in self.table.selectedIndexes()}
    if not rows or not self._results:
        return
    idx = min(rows)
    if idx < len(self._results):
        self.show_result(self._results[idx])
def clear(self) -> None:
    self._results.clear()
    self.table.setRowCount(0)
    self.view_orig.clear_image()
    self.view_heat.clear_image()
    self.txt_report.clear()
def results(self) -> List[object]:
    return list(self._results)
@property
def summary_counts(self) -> Dict[str, int]:
    fake = sum(1 for r in self._results if r.is_fake)
    return {"fake": fake, "real": len(self._results) - fake, "total": len(self._results)}
/* ===== 文件: app/modules/settings_panel.py ===== */
"""桌面工具 · 系统设置模块
可配置项（申报书 3.2）：推理后端、推理设备、检测阈值、结果保存路径、模型路径。
所有配置通过 QSettings 持久化到注册表/ini，下次启动自动恢复。
"""

from __future__ import annotations
import os
from PyQt5.QtCore import QSettings, pyqtSignal
from PyQt5.QtWidgets import (
    QComboBox, QDoubleSpinBox, QFileDialog, QFormLayout, QGroupBox, QHBoxLayout,
    QLabel, QLineEdit, QPushButton, QSpinBox, QVBoxLayout, QWidget,
)

class SettingsPanel(QGroupBox):
    """⑤ 系统设置模块。"""
    applied = pyqtSignal(dict)
    DEFAULTS = {
        "backend": "torch",
        "device": "cpu",
        "threshold": 0.5,
        "save_dir": "./outputs",
        "ckpt": "checkpoints/vibnet_best.pt",
        "onnx": "deploy/vibnet_simplified.onnx",
        "openvino": "deploy/vibnet_ir/vibnet.xml",
        "config": "configs/default.yaml",
        "image_size": 224,
        "auto_load": True,
    }

    def __init__(self, parent=None, project_root: str = "."):
        super().__init__("⑤ 系统设置", parent)
        self.root = project_root

```

```
self.settings = QSettings("GXU", "VIBNetDetector")
self._build()
self.load_config()
# -----
def _build(self) -> None:
    root = QVBoxLayout(self)
    form = QFormLayout()
    self.cmb_backend = QComboBox()
    try:
        from src.deploy.inference import list_available_backends
        self.cmb_backend.addItem(list_available_backends())
    except Exception:
        self.cmb_backend.addItem(["torch", "onnx", "openvino"])
    self.cmb_backend.setToolTip("torch=调试用; onnx=通用; openvino=Intel CPU 加速 (最快)")
    form.addRow("推理后端", self.cmb_backend)
    self.cmb_device = QComboBox()
    self.cmb_device.addItem(["cpu", "gpu"])
    form.addRow("推理设备", self.cmb_device)
    self.spin_thr = QDoubleSpinBox()
    self.spin_thr.setRange(0.05, 0.95)
    self.spin_thr.setSingleStep(0.05)
    self.spin_thr.setDecimals(2)
    self.spin_thr.setToolTip("伪造概率大于该阈值即判定为疑似伪造, 默认 0.50")
    form.addRow("检测阈值", self.spin_thr)
    self.spin_size = QSpinBox()
    self.spin_size.setRange(128, 512)
    self.spin_size.setSingleStep(32)
    form.addRow("输入尺寸", self.spin_size)
    self.edit_save = QLineEdit()
    btn_save = QPushButton("...")
    btn_save.setFixedWidth(30)
    row = QHBoxLayout()
    row.addWidget(self.edit_save)
    row.addWidget(btn_save)
    w = QWidget()
    w.setLayout(row)
    form.addRow("结果保存路径", w)
    btn_save.clicked.connect(lambda: self._pick_dir(self.edit_save))
    self.edit_ckpt = QLineEdit()
    b1 = QPushButton("...")
    b1.setFixedWidth(30)
    r1 = QHBoxLayout(); r1.addWidget(self.edit_ckpt); r1.addWidget(b1)
    w1 = QWidget(); w1.setLayout(r1)
    form.addRow("PyTorch 权重", w1)
    b1.clicked.connect(lambda: self._pick_file(self.edit_ckpt, "*.pt"))
    self.edit_onnx = QLineEdit()
    b2 = QPushButton("...")
    b2.setFixedWidth(30)
    r2 = QHBoxLayout(); r2.addWidget(self.edit_onnx); r2.addWidget(b2)
    w2 = QWidget(); w2.setLayout(r2)
    form.addRow("ONNX 模型", w2)
    b2.clicked.connect(lambda: self._pick_file(self.edit_onnx, "*.onnx"))
```

```

        self.edit_ov = QLineEdit()
        b3 = QPushButton("...")
        b3.setFixedWidth(30)
        r3 = QHBoxLayout(); r3.addWidget(self.edit_ov); r3.addWidget(b3)
        w3 = QWidget(); w3.setLayout(r3)
        form.addRow("OpenVINO IR", w3)
        b3.clicked.connect(lambda: self._pick_file(self.edit_ov, "*.xml"))
        root.addLayout(form)
        btns = QHBoxLayout()
        self.btn_apply = QPushButton("保存并加载模型")
        self.btn_reset = QPushButton("恢复默认")
        btns.addWidget(self.btn_apply)
        btns.addWidget(self.btn_reset)
        btns.addStretch(1)
        root.addLayout(btns)
        self.lbl_status = QLabel("就绪")
        self.lbl_status.setStyleSheet("color:#666;")
        root.addWidget(self.lbl_status)
        self.btn_apply.clicked.connect(self._apply)
        self.btn_reset.clicked.connect(self.reset)

# -----
def _pick_dir(self, edit: QLineEdit) -> None:
    d = QFileDialog.getExistingDirectory(self, "选择目录", edit.text() or self.root)
    if d:
        edit.setText(d)
def _pick_file(self, edit: QLineEdit, pattern: str) -> None:
    p, _ = QFileDialog.getOpenFileName(self, "选择文件", self.root, f"文件 ({pattern})")
    if p:
        edit.setText(p)

# -----
def get_config(self) -> dict:
    return {
        "backend": self.cmb_backend.currentText(),
        "device": self.cmb_device.currentText(),
        "threshold": float(self.spin_thr.value()),
        "save_dir": self.edit_save.text().strip() or "./outputs",
        "ckpt": self.edit_ckpt.text().strip(),
        "onnx_path": self.edit_onnx.text().strip(),
        "openvino_ir": self.edit_ov.text().strip(),
        "image_size": int(self.spin_size.value()),
        "config_path": self._abs(self.DEFAULTS["config"]),
    }
def _abs(self, p: str) -> str:
    return p if os.path.isabs(p) else os.path.normpath(os.path.join(self.root, p))
def _apply(self) -> None:
    cfg = self.get_config()
    for k, v in cfg.items():
        self.settings.setValue(k, v)
    self.applied.emit(cfg)
def reset(self) -> None:
    for k, v in self.DEFAULTS.items():
        self.settings.setValue(k, v)

```

```

        self.load_config()
        self.lbl_status.setText("已恢复默认设置")
def load_config(self) -> None:
    for k, dv in self.DEFAULTS.items():
        v = self.settings.value(k, dv)
        if k == "backend":
            i = self.cmb_backend.findText(str(v))
            self.cmb_backend.setCurrentIndex(i if i >= 0 else 0)
        elif k == "device":
            i = self.cmb_device.findText(str(v))
            self.cmb_device.setCurrentIndex(i if i >= 0 else 0)
        elif k == "threshold":
            self.spin_thr.setValue(float(v))
        elif k == "image_size":
            self.spin_size.setValue(int(v))
        elif k == "save_dir":
            self.edit_save.setText(str(v))
        elif k == "ckpt":
            self.edit_ckpt.setText(str(v))
        elif k == "onnx":
            self.edit_onnx.setText(str(v))
        elif k == "openvino":
            self.edit_ov.setText(str(v))
def set_status(self, text: str, color: str = "#666") -> None:
    self.lbl_status.setText(text)
    self.lbl_status.setStyleSheet(f"color: {color};")
/* ===== 文件: app/modules/exporter.py ===== */
"""桌面工具 • 结果导出模块
导出内容（申报书 3.2）：
    ① PDF 检测报告：原始图像 + 篡改区域高亮图 + 检测结果信息
    ② PNG 篡改掩码图：白色=篡改区域，黑色=真实区域
PDF 生成方案：优先使用 PyQt5 自带的 QtPrintSupport (QTextDocument -> QPrinter)，
零额外依赖；未安装 QtPrintSupport 时回退到 reportlab。
"""

from __future__ import annotations
import html
import os
import tempfile
import time
from typing import List, Optional
import numpy as np
def _save_png_cn(img: np.ndarray, path: str) -> str:
    """保存 PNG，兼容中文路径。"""
    import cv2
    ok, buf = cv2.imencode(".png", img)
    if not ok:
        raise RuntimeError(f"编码失败: {path}")
    buf.tofile(path)
    return path
def export_mask_png(mask: np.ndarray, path: str) -> str:
    """导出篡改掩码 PNG（白=篡改，黑=真实）。"""
    binary = (np.clip(mask, 0, 1) > 0.5).astype(np.uint8) * 255

```

```
    if not path.lower().endswith(".png"):
        path += ".png"
    return _save_png_cn(binary, path)
def export_heatmap_png(overlay_bgr: np.ndarray, path: str) -> str:
    if not path.lower().endswith(".png"):
        path += ".png"
    return _save_png_cn(overlay_bgr, path)
# =====
def _build_html(results: List[object], title: str = "AI 图像篡改检测报告") -> str:
    rows = []
    n_fake = sum(1 for r in results if r.is_fake)
    for i, r in enumerate(results, 1):
        color = "#c0392b" if r.is_fake else "#1e8e3e"
        rows.append(
            f"<tr><td>{i}</td><td>{html.escape(r.name)}</td>"
            f"<td>{r.width}x{r.height}</td>"
            f"<td style='color:{color};font-weight:bold'>{r.label}</td>"
            f"<td>{r.prob_fake:.4f}</td>"
            f"<td>{float((r.mask > 0.5).mean()):.4f}</td>"
            f"<td>{r.time_ms:.1f}</td></tr>"
        )
    detail = []
    for i, r in enumerate(results[:20], 1):  # 详细页最多 20 张, 避免报告过长
        detail.append(f"""
<div style="margin-top:14px;">
    <div style="font-weight:bold;color:('{#c0392b' if r.is_fake else '#1e8e3e'});">
        {i}. {html.escape(r.name)} —— {r.label} (伪造置信度 {r.prob_fake:.4f})
    </div>
</div>""")
    return f"""<html><head><meta charset="utf-8"></head><body
    style="font-family:'Microsoft YaHei',sans-serif;font-size:11pt;">
<h2 style="margin-bottom:4px;">{title}</h2>
<p style="color:#666;margin-top:0;">
    生成时间: {time.strftime('%Y-%m-%d %H:%M:%S')} |
    受检图像 {len(results)} 张, 其中疑似伪造 {n_fake} 张, 判定为真实 {len(results)-n_fake} 张
</p>
<table border="1" cellspacing="0" cellpadding="5"
    style="border-collapse:collapse;width:100%;font-size:10pt;">
<tr style="background:#eef1f6;">
    <th>#</th><th>文件名</th><th>尺寸</th><th>结论</th>
    <th>伪造置信度</th><th>可疑区域占比</th><th>耗时(ms)</th>
</tr>
{''.join(rows)}
</table>
<p style="color:#999;font-size:9pt;margin-top:16px;">
    说明: 结论由本地运行的轻量 VIB-Net 模型自动给出, 仅供辅助参考,
    不作为司法鉴定依据。本报告全部在本地生成, 图像数据未离开本机。
</p>
</body></html>"""
def export_pdf_report(results: List[object], path: str,
    title: str = "AI 图像篡改检测报告",
    include_images: bool = True) -> str:
    """导出 PDF 检测报告。"""
```

```

    if not results:
        raise ValueError("没有可导出的检测结果")
    if not path.lower().endswith(".pdf"):
        path += ".pdf"
    body = _build_html(results, title)
    try:
        return _export_pdf_qt(results, path, body, include_images)
    except Exception as e:  # noqa: BLE001
        print(f"[export] Qt 导出 PDF 失败 ({e}), 尝试 reportlab")
        return _export_pdf_reportlab(results, path, title, include_images)
def _export_pdf_qt(results: List[object], path: str, body: str,
                    include_images: bool) -> str:
    from PyQt5.QtCore import QUrl
    from PyQt5.QtGui import QTextDocument
    from PyQt5.QtPrintSupport import QPrinter
    from PyQt5.QtWidgets import QApplication
    app = QApplication.instance()
    if app is None:
        raise RuntimeError("需要先创建 QApplication")
    tmpdir = tempfile.mkdtemp(prefix="vibnet_report_")
    img_html = ""
    if include_images:
        for i, r in enumerate(results[:20], 1):  # 图录限 20 张
            from app.modules.result_panel import read_image_bgr
            orig = read_image_bgr(r.path)
            if orig is None:
                continue
            op = os.path.join(tmpdir, f"{i}_orig.png").replace("\\", "/")
            hp = os.path.join(tmpdir, f"{i}_heat.png").replace("\\", "/")
            _save_png_cn(orig, op)
            _save_png_cn(r.overlay, hp)
            img_html += f"""
<div style="margin-top:16px;">
    <div style="font-size:10pt;font-weight:bold;">
        {i}. {html.escape(r.name)} —— {r.label} ({r.prob_fake:.4f})
    </div>
    <table style="width:100%;margin-top:6px;"><tr>
        <td style="width:50%;"></td>
        <td style="width:50%;"></td>
    </tr>
    <tr style="font-size:9pt;color:#666;">
        <td align="center">原始图像</td><td align="center">篡改区域高亮</td>
    </tr></table>
</div>"""
    doc = QTextDocument()
    doc.setHtml(body.replace("</body>", img_html + "</body>"))
    printer = QPrinter(QPrinter.HighResolution)
    printer.setOutputFormat(QPrinter.PdfFormat)
    printer.setOutputFileName(path)
    doc.print_(printer)
    return path
def _export_pdf_reportlab(results: List[object], path: str, title: str,

```

```

        include_images: bool) -> str:
    """reportlab 回退实现。"""
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.units import mm
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.cidfonts import UnicodeCIDFont
    from reportlab.pdfgen import canvas
    try:
        pdfmetrics.registerFont(UnicodeCIDFont("STSong-Light"))
        font = "STSong-Light"
    except Exception:
        font = "Helvetica"
    c = canvas.Canvas(path, pagesize=A4)
    w, h = A4
    y = h - 20 * mm
    c.setFont(font, 15)
    c.drawString(20 * mm, y, title)
    y -= 8 * mm
    c.setFont(font, 9)
    c.drawString(20 * mm, y, f"生成时间: {time.strftime('%Y-%m-%d %H:%M:%S')} "
                  f"受检 {len(results)} 张")
    y -= 8 * mm
    c.setFont(font, 9)
    for i, r in enumerate(results, 1):
        if y < 30 * mm:
            c.showPage()
            c.setFont(font, 9)
            y = h - 20 * mm
            c.drawString(20 * mm, y, f"{i}. {r.name} {r.width}x{r.height} "
                  f"{r.label} p={r.prob_fake:.4f} "
                  f"{(r.mask > 0.5).mean():.4f} {r.time_ms:.1f}ms")
            y -= 6 * mm
        c.save()
    return path
def export_batch_masks(results: List[object], out_dir: str) -> List[str]:
    """批量导出掩码 PNG 到指定目录。"""
    os.makedirs(out_dir, exist_ok=True)
    out = []
    for r in results:
        base = os.path.splitext(r.name)[0]
        p = os.path.join(out_dir, f"{base}_mask.png")
        out.append(export_mask_png(r.mask, p))
    return out
/* ===== 文件: app/modules/workers.py ===== */
"""桌面工具 • 检测处理模块 —— 异步推理工作线程
设计要点（申报书 3.2 "检测处理模块：加载优化后的模型，实现异步推理，避免界面卡顿"）：
* 模型加载与推理都放在 QThread 中，主线程（UI）只负责信号槽通信；
* 批量检测支持中途取消（`requestInterruption`）；
* 逐张 emit 结果，界面可以边检测边刷新，长任务也有实时反馈。
"""
from __future__ import annotations
import os

```

```

import traceback
from typing import List, Optional
from PyQt5.QtCore import QThread, pyqtSignal
class DetectorLoader(QThread):
    """后台加载模型（首次加载 CLIP 权重较慢，不能阻塞 UI）。"""
    loaded = pyqtSignal(object)      # Detector
    failed = pyqtSignal(str)
    def __init__(self, kwargs: dict, parent=None):
        super().__init__(parent)
        self.kwargs = kwargs
    def run(self) -> None:
        try:
            from src.deploy.inference import Detector
            det = Detector(**self.kwargs)
            self.loaded.emit(det)
        except Exception:
            self.failed.emit(traceback.format_exc())
class BatchDetectWorker(QThread):
    """批量检测工作线程。"""
    progress = pyqtSignal(int, int, str)      # 当前索引, 总数, 文件名
    one_done = pyqtSignal(object)            # DetectionResult
    finished_all = pyqtSignal(list, list)     # (成功列表, 失败列表[(path, err)])
    failed = pyqtSignal(str)
    def __init__(self, detector, paths: List[str], parent=None):
        super().__init__(parent)
        self.detector = detector
        self.paths = list(paths)
    def run(self) -> None:
        if self.detector is None:
            self.failed.emit("模型尚未加载完成")
            return
        ok, bad = [], []
        total = len(self.paths)
        for i, p in enumerate(self.paths):
            if self.isInterruptionRequested():
                break
            self.progress.emit(i + 1, total, os.path.basename(p))
            try:
                res = self.detector.detect(p)
                ok.append(res)
                self.one_done.emit(res)
            except Exception as e: # noqa: BLE001
                bad.append((p, f"{type(e).__name__}: {e}"))
        self.finished_all.emit(ok, bad)
/* ===== 文件: scripts/train.py ===== */
"""训练入口（三阶段分任务训练）
用法：
# 用离线演示数据集快速跑通全流程（CPU 可跑，几分钟）
python scripts/train.py --demo --epochs-scale 0.05
# 正式训练（需要三套公开数据集，建议 GPU）
python scripts/train.py --config configs/default.yaml
# 从断点继续

```

```
python scripts/train.py --resume checkpoints/vibnet_last.pt
```

常用参数:

```
--epochs-scale 按比例缩放各阶段轮数 (0.05 表示 70 轮 -> 约 4 轮, 用于冒烟测试)
--limit-batches 限制每个 epoch 的批次次数 (进一步加速自检)
--device        cpu / cuda
--workers       DataLoader 进程数
```

```
"""
from __future__ import annotations
import argparse
import os
import sys

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
import torch # noqa: E402
from src.data.datasets import build_data_loaders # noqa: E402
from src.engine.trainer import Logger, Trainer, set_seed # noqa: E402
from src.models.vibnet import build_model, load_config # noqa: E402

def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default=os.path.join(ROOT, "configs/default.yaml"))
    ap.add_argument("--demo", action="store_true", help="使用离线演示数据集")
    ap.add_argument("--epochs-scale", type=float, default=1.0)
    ap.add_argument("--batch-size", type=int, default=None)
    ap.add_argument("--workers", type=int, default=None)
    ap.add_argument("--device", default=None)
    ap.add_argument("--resume", default=None)
    ap.add_argument("--limit-batches", type=int, default=None)
    ap.add_argument("--tag", default="vibnet")
    args = ap.parse_args()
    cfg = load_config(args.config)
    if args.batch_size:
        cfg["train"]["batch_size"] = args.batch_size
    if args.workers is not None:
        cfg["data"]["num_workers"] = args.workers
    if args.demo:
        cfg["project"]["output_dir"] = os.path.join(ROOT, "outputs", "run_demo")
        cfg["project"]["ckpt_dir"] = os.path.join(ROOT, "checkpoints")
    if args.epochs_scale != 1.0:
        for s in cfg["train"]["stages"]:
            s["epochs"] = max(1, int(round(s["epochs"] * args.epochs_scale)))
        cfg["train"]["log_every"] = 5
        cfg["train"]["save_every"] = 999
        print(f"[train] 轮数已按 {args.epochs_scale} 缩放 -> "
              f"{[s['epochs'] for s in cfg['train']['stages']]}")
    os.makedirs(cfg["project"]["output_dir"], exist_ok=True)
    set_seed(cfg["project"].get("seed", 3407))
    dev = torch.device(args.device or ("cuda" if torch.cuda.is_available() else "cpu"))
    print(f"[train] 设备: {dev}")
    loaders = build_data_loaders(cfg, use_demo=args.demo)
    print(f"[train] 训练集 {len(loaders['train'].dataset)} 张, "
          f"验证集 {len(loaders['val'].dataset)} 张")
```

```

if args.limit_batches:
    from itertools import islice
    class _Limited:
        def __init__(self, loader, n):
            self.loader, self.n = loader, n
        def __iter__(self):
            return islice(iter(self.loader), self.n)
        def __len__(self):
            return min(len(self.loader), self.n)
    loaders["train"] = _Limited(loaders["train"], args.limit_batches)
    loaders["val"] = _Limited(loaders["val"], max(1, args.limit_batches // 2))
model = build_model(cfg)
for k, v in model.summary().items():
    print(f" {k:18s} {v:.3f}" if isinstance(v, float) else f" {k:18s} {v}")
logger = Logger(os.path.join(cfg["project"]["output_dir"], "train_log.txt"))
trainer = Trainer(cfg, model, dev, loaders["train"], loaders["val"],
                  logger=logger, ckpt_prefix=args.tag)
if args.resume and os.path.exists(args.resume):
    trainer.load(args.resume)
trainer.fit()
print(f"\n[train] 完成。最佳 {cfg['train']['monitor']}="
      f"{trainer.best_raw:.4f} @ epoch {trainer.best_epoch}")
print(f"[train] 权重目录: {cfg['project']['ckpt_dir']}")
print(f"[train] 日志: {cfg['project']['output_dir']}/train_log.txt")
return 0
if __name__ == "__main__":
    sys.exit(main())
/* ===== 文件: scripts/fetch_datasets.py ===== */
#!/usr/bin/env python
# -*- coding: utf-8 -*-
"""ForenSynths (CNNDetection) 官方数据集的下载与解压。
数据来源（官方，托管在 HuggingFace，**无需向作者发邮件申请**）：
https://huggingface.co/datasets/sywang/CNNDetection
官方仓库 scripts 原文（dataset/{train,val,test}/download_*set.sh）：
val    -> progan_val.zip                ~0.79 GB
test   -> CNN_synth_testset.zip         ~18.7 GB   (13 种生成器，约 9 万张)
        progan_testset.zip              ~0.80 GB   (仅 ProGAN 部分)
train  -> progan_train.7z.001 ... .007  ~70.4 GB   (20 类，约 72 万张)
        7 卷合并解压得到 progan_train.zip，再解压得 train/
总积约 90 GB；解压后另需约 85~100 GB。**请预留至少 200 GB 磁盘。**
用法
----
# ① 先跑通真实数据（只下 1.6 GB，推荐第一步）
python scripts/fetch_datasets.py --split val test
# ② 全量（90 GB，建议挂一晚上）
python scripts/fetch_datasets.py --split all
# ③ 训练集只取 4 个类，省磁盘（需 7z 或 py7zr）
python scripts/fetch_datasets.py --split train --classes car cat chair horse
# ④ 换国内镜像（huggingface.co 慢时用）
python scripts/fetch_datasets.py --split val --mirror hf-mirror
# ⑤ 只校验本地已有文件，不下载
python scripts/fetch_datasets.py --split all --check-only

```

```

"""
from __future__ import annotations
import argparse
import os
import shutil
import subprocess
import sys
import time
import urllib.error
import urllib.request
import zipfile

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
# ----- 源与文件表
HF_REPO = "sywang/CNNDetection"
ENDPOINTS = {
    "hf": "https://huggingface.co",
    "hf-mirror": "https://hf-mirror.com",
}
SPLIT_FILES = {
    "val": ["progan_val.zip"],
    "progan_test": ["progan_testset.zip"],
    "test": ["CNN_synth_testset.zip"],
    "train": [f"progan_train.7z.{i:03d}" for i in range(1, 8)],
}
# 各文件解压后的目标子目录（相对 data/Datasets）。
#
# □ 官方各压缩包의**内部层级并不一致**，实测：
#   progan_val.zip      -> <class>/{0_real,1_fake}
#   progan_testset.zip -> progan/<class>/{0_real,1_fake}
#   CNN_synth_testset -> <generator>/[<class>/]{0_real,1_fake}
#   所以不能直接解压到同一个目录（会互相混层，且 val 会被 test 覆盖）。
#   统一做法：先解到 staging，再用 _move_children 归位到 train/val/test。
EXTRACT_TARGET = {
    "progan_val.zip": "ForenSynths/val",
    "progan_testset.zip": "ForenSynths/test",
    "CNN_synth_testset.zip": "ForenSynths/test",
    "progan_train.zip": "ForenSynths/train",
}
# 官方给出的体积（字节），用于校验下载完整性
EXPECTED_SIZE = {
    "progan_val.zip": 830_792_545,
    "progan_testset.zip": 834_215_577,          # 约 0.80 GB，仅作参考
    "CNN_synth_testset.zip": 20_053_000_000,    # 约 18.7 GB，仅作参考
}
UA = {"User-Agent": "Mozilla/5.0 (compatible; vibnet-dataset-fetcher/1.0)"}
# ----- 工具函数
def human(n: float) -> str:
    for unit in ("B", "KB", "MB", "GB"):
        if abs(n) < 1024:
            return f"{n:.1f} {unit}"
        n /= 1024
    return f"{n:.1f} TB"

```

```
def free_gb(path: str) -> float:
    return shutil.disk_usage(path).free / 1024 ** 3

def resolve_url(name: str, endpoint: str) -> str:
    return f"{endpoint}/datasets/{HF_REPO}/resolve/main/{name}"

def remote_size(url: str) -> int | None:
    """拿远端文件大小（跟随重定向，读 x-linked-size 或 content-length）。"""
    req = urllib.request.Request(url, headers=UA, method="HEAD")
    try:
        with urllib.request.urlopen(req, timeout=45) as r:
            for key in ("x-linked-size", "Content-Length", "content-length"):
                v = r.headers.get(key)
                if v:
                    return int(v)
    except Exception:
        return None
    return None

def download(name: str, dest_dir: str, endpoint: str, retries: int = 4) -> str | None:
    """下载单个文件，支持 HTTP Range 断点续传。返回本地路径。"""
    os.makedirs(dest_dir, exist_ok=True)
    dest = os.path.join(dest_dir, name)
    url = resolve_url(name, endpoint)
    total = remote_size(url)
    if os.path.exists(dest):
        cur = os.path.getsize(dest)
        if total and cur == total:
            print(f" [skip] {name} 已完整 ({human(cur)})")
            return dest
        if total and cur > total:
            print(f" [warn] {name} 本地比远端大，重新下载")
            os.remove(dest)
            cur = 0
        elif cur:
            print(f" [resume] {name} 从 {human(cur)} 继续")
    else:
        cur = 0
    if total:
        print(f" [get ] {name} 远端 {human(total)}"
              + (f" 剩余 {human(total - cur)}" if cur else ""))
    for attempt in range(retries):
        headers = dict(UA)
        if cur:
            headers["Range"] = f"bytes={cur}-"
        try:
            req = urllib.request.Request(url, headers=headers)
            with urllib.request.urlopen(req, timeout=90) as r, \
                open(dest, "ab" if cur else "wb") as f:
                got, last, t0 = cur, time.time(), time.time()
                while True:
                    chunk = r.read(1024 * 512)
                    if not chunk:
                        break
                    f.write(chunk)
```

---

```

        got += len(chunk)
        now = time.time()
        if now - last >= 2.0:                                # 每 2 秒刷一次进度
            speed = (got - cur) / max(now - t0, 1e-6)
            if total:
                pct = 100.0 * got / total
                eta = (total - got) / max(speed, 1.0)
                print(f"\r      {pct:5.1f}%    {human(got)}/{human(total)}    "
                      f"{human(speed)}/s    剩余 {eta/60:.1f} 分钟    ",
                      end="", flush=True)
            else:
                print(f"\r      {human(got)}    {human(speed)}/s    ",
                      end="", flush=True)
            last = now
    print()
    size = os.path.getsize(dest)
    if total and size != total:
        print(f" [warn] {name} 大小不符: {size} != {total}, 将续传")
        cur = size
        continue
    print(f" [ok ] {name}    {human(size)}")
    return dest
except (urllib.error.URLError, TimeoutError, ConnectionError, OSError) as e:
    cur = os.path.getsize(dest) if os.path.exists(dest) else 0
    print(f"\n [retry {attempt + 1}/{retries}] {type(e).__name__}: {e}")
    time.sleep(min(5 * (attempt + 1), 20))
print(f" [FAIL] {name} 多次重试仍失败")
return None
def find_7z() -> str | None:
    """按优先级寻找 7z 可执行文件（项目内置 > 系统安装 > PATH）。"""
    here = os.path.join(ROOT, "tools")
    candidates = [
        os.path.join(here, "7zr.exe"),                # 项目内置独立版（推荐，免安装）
        os.path.join(here, "7z.exe"),
        r"C:\Program Files\7-Zip\7z.exe",
        r"C:\Program Files (x86)\7-Zip\7z.exe",
        "7z", "7za", "7z.exe",
    ]
    for c in candidates:
        if os.sep in c or "/" in c:
            if os.path.exists(c):
                return c
    else:
        p = shutil.which(c)
        if p:
            return p
    return None
def ensure_7z() -> str | None:
    """确保有可用的 7z；项目内没有时自动从 7-zip.org 下载独立版 7zr.exe。"""
    exe = find_7z()
    if exe:
        return exe

```

```

url = "https://www.7-zip.org/a/7zr.exe"
dest_dir = os.path.join(ROOT, "tools")
os.makedirs(dest_dir, exist_ok=True)
dest = os.path.join(dest_dir, "7zr.exe")
print(f" [7z ] 未找到 7z, 尝试下载独立版 {url}")
req = urllib.request.Request(url, headers=UA)
try:
    with urllib.request.urlopen(req, timeout=90) as r, open(dest, "wb") as f:
        shutil.copyfileobj(r, f)
except Exception as e:
    # noqa: BLE001
    print(f" [7z ] 下载失败: {type(e).__name__}: {e}")
    return None
print(f" [7z ] 已就位 {dest} ({human(os.path.getsize(dest))})")
return dest

def extract_7z(first_volume: str, out_dir: str) -> bool:
    """用 7z 命令行解压多卷压缩包; 没有 7z 时回退到 py7zr。"""
    exe = ensure_7z()
    if exe:
        print(f" [7z ] {os.path.basename(exe)} x {os.path.basename(first_volume)}")
        r = subprocess.run([exe, "x", first_volume, f"-o{out_dir}", "-y"],
                           capture_output=True, text=True)
        if r.returncode != 0:
            print(f" [7z ] 失败: {r.stderr.strip()[:300]}")
            return False
        return True
    print(" [7z ] 未找到 7z 可执行文件, 改用 py7zr")
    try:
        import py7zr
    except ImportError:
        print(" [7z ] py7zr 未安装. 请执行其一: ")
        print("         pip install py7zr")
        print("         或安装 7-Zip 后把 7z.exe 加进 PATH")
        return False
    with py7zr.SevenZipFile(first_volume, "r") as z:
        z.extractall(path=out_dir)
    return True

def extract_zip(archive: str, out_dir: str, classes: list[str] | None = None) -> int:
    """解压 zip。classes 非空时只解包含这些类的成员 (用于训练集瘦身)。"""
    os.makedirs(out_dir, exist_ok=True)
    n = 0
    with zipfile.ZipFile(archive) as z:
        names = z.namelist()
        members = names
        if classes:
            keys = tuple(f"/{c}/" for c in classes)
            members = [m for m in names
                       if m.endswith("/") or any(k in f"/{m}" for k in keys)]
            print(f" [zip ] 选择性解压 {len(members)}/{len(names)} 项"
                  f" (类: {' '.join(classes)})")
        else:
            print(f" [zip ] 解压 {len(names)} 项")
        for i, m in enumerate(members, 1):

```

```

        try:
            z.extract(m, out_dir)
            n += 1
        except Exception as e:
            # noqa: BLE001
            print(f" [warn] {m}: {type(e).__name__}: {e}")
        if i % 5000 == 0:
            print(f"\r    已解压 {i}/{len(members)}    ", end="", flush=True)
        print()
    return n

def _move_children(src_dir: str, dst_dir: str,
                  strip: tuple = ("train", "val", "test")) -> int:
    """把 src_dir 下的条目搬进 dst_dir。
    若唯一顶层目录名恰好是 train/val/test, 则先剥掉这一层再搬
    (官方压缩包内部有时带一层 split 目录, 有时不带, 两种都要能吃)。
    目标已存在同名条目时**跳过并告警, 绝不覆盖**已有数据。
    """
    os.makedirs(dst_dir, exist_ok=True)
    entries = [e for e in os.listdir(src_dir) if not e.startswith(".")]
    if len(entries) == 1:
        only = os.path.join(src_dir, entries[0])
        if entries[0] in strip and os.path.isdir(only):
            src_dir = only
            entries = [e for e in os.listdir(src_dir) if not e.startswith(".")]
    moved = skipped = 0
    for e in entries:
        s, d = os.path.join(src_dir, e), os.path.join(dst_dir, e)
        if os.path.exists(d):
            print(f" [skip] {e} 已存在, 不覆盖")
            skipped += 1
            continue
        shutil.move(s, d)
        moved += 1
    if skipped:
        print(f" [warn] {skipped} 项因目标已存在被跳过")
    return moved

# ----- 主流程
def main() -> int:
    ap = argparse.ArgumentParser(
        description="下载 ForenSynths(CNNDetection) 官方数据集",
        formatter_class=argparse.RawDescriptionHelpFormatter,
    )
    ap.add_argument("--root", default=os.path.join(ROOT, "data"),
                    help="数据根目录, 默认 <项目>/data")
    ap.add_argument("--split", nargs="+", default=["val"],
                    choices=list(SPLIT_FILES) + ["all"],
                    help="要下载的划分, 可多选, all=全部 (约 90GB) ")
    ap.add_argument("--mirror", default="hf", choices=list(ENDPOINTS),
                    help="下载源: hf(默认) | hf-mirror(hf-mirror.com)")
    ap.add_argument("--classes", nargs="+", default=None,
                    help="只解压训练集里这些类, 如 car cat chair horse")
    ap.add_argument("--keep-archive", action="store_true",
                    help="解压后保留压缩包 (默认删除以省磁盘)")

```

```
ap.add_argument("--check-only", action="store_true",
                help="只检查本地文件, 不下载")
ap.add_argument("--no-extract", action="store_true", help="只下载不解压")
args = ap.parse_args()
dl_dir = os.path.join(args.root, "_downloads")
ds_dir = os.path.join(args.root, "Datasets")
os.makedirs(dl_dir, exist_ok=True)
os.makedirs(ds_dir, exist_ok=True)
splits = list(SPLIT_FILES) if "all" in args.split else args.split
todo: list[str] = []
for s in splits:
    for f in SPLIT_FILES[s]:
        if f not in todo:
            todo.append(f)
endpoint = ENDPOINTS[args.mirror]
print("=" * 68)
print(f"数据源      : {endpoint}/datasets/{HF_REPO}")
print(f"下载目录    : {dl_dir}")
print(f"数据目录    : {ds_dir}")
print(f"待下载      : {len(todo)} 个文件")
print(f"磁盘可用    : {free_gb(args.root):.1f} GB")
print("=" * 68)
if args.check_only:
    for f in todo:
        p = os.path.join(dl_dir, f)
        if os.path.exists(p):
            print(f"  [有] {f}  {human(os.path.getsize(p))}")
        else:
            print(f"  [缺] {f}")
    return 0
need = 0
for f in todo:
    p = os.path.join(dl_dir, f)
    have = os.path.getsize(p) if os.path.exists(p) else 0
    if f.endswith(".7z.001"):
        need += 70.4 * 1024 ** 3 - have
    elif f.startswith("CNN_synth"):
        need += 18.7 * 1024 ** 3 - have
    else:
        need += 0.8 * 1024 ** 3 - have
need_gb = max(need, 0) / 1024 ** 3
avail = free_gb(args.root)
if need_gb > avail * 0.9:
    print(f"[警告] 预计还需 {need_gb:.1f} GB, 磁盘仅剩 {avail:.1f} GB。")
    print("      建议加 --classes car cat chair horse 分批处理, 或清理后重跑。")
    if need_gb > avail:
        print("[中止] 空间不足。")
        return 2
else:
    print(f"[磁盘] 预计还需 {need_gb:.1f} GB, 可用 {avail:.1f} GB, 充足。\\n")
ok, fail = [], []
for f in todo:
```

```

    p = download(f, dl_dir, endpoint)
    (ok if p else fail).append(f)
if args.no_extract:
    print("\n[--no-extract] 跳过解压。")
    return 0 if not fail else 1
# ---- 解压: 先解到 staging, 再规范化搬入目标目录 (见 EXTRACT_TARGET 处的说明)
print("\n" + "=" * 68)
print("开始解压")
print("=" * 68)
for f in ok:
    p = os.path.join(dl_dir, f)
    target_rel = EXTRACT_TARGET.get(f, "ForenSynths")
    target = os.path.join(ds_dir, target_rel)
    selective = args.classes if f.startswith("CNN_synth") else None
    if f.endswith(".7z.001"):
        if not extract_7z(p, dl_dir):
            fail.append(f)
            continue
        inner = os.path.join(dl_dir, "progan_train.zip")
        if not os.path.exists(inner):
            print(" [warn] 未找到解压产物 progan_train.zip")
            continue
        archives = [inner]
    else:
        archives = [p]
    for arc in archives:
        staging = os.path.join(dl_dir, "_extract",
                               os.path.splitext(os.path.basename(arc))[0])
        if os.path.isdir(staging):
            shutil.rmtree(staging)
        os.makedirs(staging, exist_ok=True)
        extract_zip(arc, staging, selective)
        n = _move_children(staging, target)
        print(f" [put ] {n} 项归位到 {target_rel}/")
        shutil.rmtree(staging, ignore_errors=True)
    if not args.keep_archive:
        if os.path.exists(p):
            os.remove(p)
        if f.endswith(".7z.001"):
            for i in range(1, 8):
                v = os.path.join(dl_dir, f"progan_train.7z.{i:03d}")
                if os.path.exists(v):
                    os.remove(v)
            inner = os.path.join(dl_dir, "progan_train.zip")
            if os.path.exists(inner):
                os.remove(inner)
            print(" [del ] progan_train.7z.001~007 + progan_train.zip (已解压)")
        else:
            print(f" [del ] {f} (已解压, 省磁盘)")
print("\n" + "=" * 68)
print(f"完成: 成功 {len(ok)} / 失败 {len(fail)}")
if fail:

```

```

        print("失败文件: " + ", ".join(fail))
    print(f"磁盘剩余: {free_gb(args.root):.1f} GB")
    print("\n下一步: 检查目录结构是否正确")
    print(f"  python scripts/prepare_datasets.py --root {args.root}/Datasets --check")
    print("=" * 68)
    return 0 if not fail else 1
if __name__ == "__main__":
    sys.exit(main())
/* ===== 文件: scripts/prepare_datasets.py ===== */
"""数据集获取与规范化组织（申报书 3.3 / 6.3 测试数据集）
三个公开标准数据集：
① ForenSynths    —— 跨生成模型整图真伪；20 类约 72 万张训练图
                    （4 类子集约 14.4 万张），验证 8000，测试约 9 万张（13 种生成器）
② CASIA v2       —— 传统篡改（拼接 / 复制移动 / 修图），7491 真 + 5123 篡改（合计 12614）
③ COVERAGE       —— 复制移动细粒度定位，100 真 + 100 篡改
本脚本做三件事：
1) 检查各数据集的落盘情况，打印缺失项与官方获取渠道；
2) 把原始布局**规范化**为项目统一布局（见 src/data/datasets.py 顶部注释）；
3) 划分 train/val/test 并输出 manifest.json（记录每条的来源、标签、掩码路径），
   便于论文里写"数据划分可复现"。
用法：
# 下载（另有独立脚本，支持断点续传与镜像）
python scripts/fetch_datasets.py --split val progan_test
# 检查 / 规范化 / 生成清单
python scripts/prepare_datasets.py --root data/Datasets --check
python scripts/prepare_datasets.py --root data/Datasets --organize --mode link
python scripts/prepare_datasets.py --root data/Datasets --manifest
注意：
* 各数据集版权归原作者所有，仅可用于学术研究；请遵守各自 License。
* ForenSynths **无需向作者申请**，官方已托管 HuggingFace 公开下载：
  https://huggingface.co/datasets/sywang/CNNDetection
本项目用 scripts/fetch_datasets.py 拉取（断点续传 + hf-mirror 镜像 + 选择性解压）。
（2026.09.18 更正：早期注释里"需要向作者申请"约 10 万张"均为错误信息。）
"""

from __future__ import annotations
import argparse
import json
import os
import random
import shutil
import sys
from glob import glob
from typing import Dict, List
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
IMG_EXT = ("*.png", "*.jpg", "*.jpeg", "*.bmp", "*.webp", "*.tif", "*.tiff")
SOURCES = {
    "ForenSynths": {
        "desc": "跨生成模型整图真伪；20 类约 72 万张训练（4 类子集约 14.4 万），"
        "验证 8000，测试约 9 万张",
        "source": "https://huggingface.co/datasets/sywang/CNNDetection "
    }
}

```

```

        "（官方公开，无需申请；用 scripts/fetch_datasets.py 下载）",
        # 训练/验证: <类别>/{0_real,1_fake}/ ; 测试: <生成器>/[<类别>/{0_real,1_fake}/
        "layout": "{root}/ForenSynths/<class>/{0_real,1_fake}/ "
        "+ {root}/ForenSynths_test/<generator>/[<class>/{0_real,1_fake}/",
        "download": "python scripts/fetch_datasets.py --split val progan_test test train",
    },
    "CASIAv2": {
        "desc": "传统篡改: 拼接 / 复制移动 / 修图 (7491 真 + 5123 篡改, 共 12614; "
        "篡改 = 3274 复制移动 + 1849 拼接)",
        "source": "GitHub: namtpham/casia2groundtruth (推荐, 标注已修正); "
        "或 Kaggle 镜像 (注意部分版本不含 GT 掩码, 做不了定位); "
        "或 CASIA v2.0 官方网站",
        "layout": "{root}/CASIAv2/<split>/{image,mask}/",
    },
    "COVERAGE": {
        "desc": "复制移动细粒度定位 (100 真 + 100 篡改)",
        "source": "https://github.com/wjct1/COVERAGE (掩码后缀随版本为 "
        " *_forged.tif 或 *_gt.png, 落盘后先确认)",
        "layout": "{root}/COVERAGE/<split>/{image,mask}/",
    },
}

SPLIT_RATIO = {"train": 0.8, "val": 0.1, "test": 0.1}

def _list_images(folder: str) -> List[str]:
    out: List[str] = []
    if not os.path.isdir(folder):
        return out
    for e in IMG_EXT:
        out += glob(os.path.join(folder, e)) + glob(os.path.join(folder, e.upper()))
    return sorted(set(out))

def _relink(src: str, dst: str, mode: str) -> None:
    os.makedirs(os.path.dirname(dst), exist_ok=True)
    if os.path.exists(dst):
        return
    if mode == "link":
        try:
            os.link(src, dst)
            return
        except OSError:
            pass
    if mode == "symlink":
        try:
            os.symlink(src, dst)
            return
        except OSError:
            pass
    shutil.copy2(src, dst)

# =====
def _scan_real_fake(base_dir: str, root: str, counts: Dict[str, int]) -> None:
    """自动探测任意层级下的 0_real / 1_fake 配对目录 (官方布局层级不固定)。"""
    if not os.path.isdir(base_dir):
        return
    for real in glob(os.path.join(base_dir, "**", "0_real"), recursive=True):

```

```

        if not os.path.isdir(real):
            continue
        rel = os.path.relpath(os.path.dirname(real), root).replace("\\", "/")
        n_real = len(_list_images(real))
        if n_real:
            counts[f"{rel}/0_real"] = n_real
        fake = os.path.join(os.path.dirname(real), "1_fake")
        n_fake = len(_list_images(fake))
        if n_fake:
            counts[f"{rel}/1_fake"] = n_fake
def check(root: str) -> Dict[str, dict]:
    print(f"\n数据集根目录: {os.path.abspath(root)}\n" + "-" * 70)
    report = {}
    for name, meta in SOURCES.items():
        base = os.path.join(root, name)
        exists = os.path.isdir(base)
        counts: Dict[str, int] = {}
        if name == "ForenSynths":
            # 官方布局层级不固定(训练/验证按类别, 测试按生成器), 全量递归探测
            for d in (os.path.join(root, "ForenSynths"),
                      os.path.join(root, "ForenSynths_test")):
                _scan_real_fake(d, root, counts)
        elif exists:
            for split in ("train", "val", "test"):
                n = len(_list_images(os.path.join(base, split, "image")))
                if n:
                    counts[f"{split}/image"] = n
            total = sum(counts.values())
            status = "已就绪" if total > 0 else ("目录存在但为空" if exists else "缺失")
            report[name] = {"status": status, "counts": counts, "total": total}
            print(f"【{name}】 {status} {meta['desc']}")
            print(f"      获取: {meta['source']}")
            if "download" in meta:
                print(f"      下载: {meta['download']}")
            # 布局串里含 {0_real,1_fake} 这类字面花括号, 不能用 str.format, 改用 replace
            print(f"      期望布局: {meta['layout'].replace('{root}', root)}")
            if counts:
                print(f"      现有 {len(counts)} 个子集, 合计 {total} 张:")
                for k in sorted(counts)[:12]:
                    print(f"          {k} / {counts[k]}")
                if len(counts) > 12:
                    print(f"          ... (另有 {len(counts) - 12} 个子集未列出)")
            print()
        return report
# =====
def split_tamper_dataset(base: str, mode: str = "link", seed: int = 3407) -> None:
    """把 CASIAv2 / COVERAGE 的 image+mask 平铺目录划分成 train/val/test."""
    src_img = os.path.join(base, "image")
    src_mask = os.path.join(base, "mask")
    if not os.path.isdir(src_img):
        return
    imgs = _list_images(src_img)

```

```
if not imgs:
    return
rng = random.Random(seed)
# 按"是否篡改"分层抽样, 保证各 split 的正负比例一致
pos, neg = [], []
for p in imgs:
    stem = os.path.splitext(os.path.basename(p))[0]
    mp = next((os.path.join(src_mask, stem + e) for e in (".png", ".jpg", ".bmp"))
              if os.path.exists(os.path.join(src_mask, stem + e))), None)
    m_found = False
    if mp:
        try:
            import cv2
            m = cv2.imread(mp, cv2.IMREAD_GRAYSCALE)
            m_found = m is not None and (m > 127).any()
        except Exception:
            m_found = True
    (pos if m_found else neg).append((p, mp))
rng.shuffle(pos)
rng.shuffle(neg)
for tag, items in (("pos", pos), ("neg", neg)):
    n = len(items)
    n_tr = int(n * SPLIT_RATIO["train"])
    n_va = int(n * SPLIT_RATIO["val"])
    buckets = {"train": items[:n_tr],
               "val": items[n_tr:n_tr + n_va],
               "test": items[n_tr + n_va:]}
    for split, sub in buckets.items():
        for p, mp in sub:
            stem = os.path.splitext(os.path.basename(p))[0]
            _relink(p, os.path.join(base, split, "image", os.path.basename(p)), mode)
            if mp:
                _relink(mp, os.path.join(base, split, "mask",
                                           os.path.basename(mp)), mode)
            else:
                # 真实图像没有掩码文件 -> 生成全黑掩码, 保持"每条都有 mask"
                import numpy as np
                import cv2
                out = os.path.join(base, split, "mask", stem + ".png")
                if not os.path.exists(out):
                    os.makedirs(os.path.dirname(out), exist_ok=True)
                    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
                    h, w = img.shape if img is not None else (256, 256)
                    cv2.imwrite(out, np.zeros((h, w), np.uint8))
def organize(root: str, mode: str = "link") -> None:
    print(f"\n规范化组织数据集 (mode={mode}) ")
    for name in ("CASIAv2", "COVERAGE"):
        base = os.path.join(root, name)
        if os.path.isdir(base):
            split_tamper_dataset(base, mode)
            print(f"    [ok] {name} 已划分 train/val/test")
    foren = os.path.join(root, "ForenSynths")
```

```

    if os.path.isdir(foren):
        # 官方布局 <generator>/<split>/{0_real,1_fake} 已经能被 GenSynthsDataset 直接读取,
        # 这里只做一致性检查与合并 (可选)。
        n = 0
        for p in glob(os.path.join(foren, "*", "*", "0_real")):
            n += len(_list_images(p))
        print(f" [ok] ForenSynths 检测到 {n} 张真实图 (官方层级可直接使用)")
# =====
def manifest(root: str, out: str = "") -> str:
    """生成 manifest.json: 每条样本的路径 / 标签 / 掩码 / 来源 / split。"""
    recs = []
    for name in ("ForenSynths", "CASIAv2", "COVERAGE"):
        base = os.path.join(root, name)
        if not os.path.isdir(base):
            continue
        for split in ("train", "val", "test"):
            img_dir = os.path.join(base, split, "image")
            if os.path.isdir(img_dir):
                mask_dir = os.path.join(base, split, "mask")
                for p in _list_images(img_dir):
                    stem = os.path.splitext(os.path.basename(p))[0]
                    mp = next((os.path.join(mask_dir, stem + e)
                                for e in (".png", ".jpg", ".bmp")
                                if os.path.exists(os.path.join(mask_dir, stem + e))), None)
                    label = 0
                    if mp:
                        import cv2
                        m = cv2.imread(mp, cv2.IMREAD_GRAYSCALE)
                        label = int(m is not None and (m > 127).any())
                    recs.append({"path": os.path.relpath(p, root), "label": label,
                                "mask": os.path.relpath(mp, root) if mp else None,
                                "source": name, "split": split, "task": "tamper"})
                continue
        for cls, lab in (("0_real", 0), ("1_fake", 1)):
            for p in _list_images(os.path.join(base, split, cls)):
                recs.append({"path": os.path.relpath(p, root), "label": lab,
                            "mask": None, "source": name, "split": split,
                            "task": "gensynth"})
    out = out or os.path.join(root, "manifest.json")
    os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
    with open(out, "w", encoding="utf-8") as f:
        json.dump({"root": os.path.abspath(root), "n": len(recs), "records": recs},
                  f, ensure_ascii=False, indent=2)
    stats: Dict[str, Dict[str, int]] = {}
    for r in recs:
        k = f"{r['source']}/{r['split']}"
        stats.setdefault(k, {"total": 0, "fake": 0})
        stats[k]["total"] += 1
        stats[k]["fake"] += r["label"]
    print(f"\nmanifest 已写入 {out} (共 {len(recs)} 条)")
    for k, v in sorted(stats.items()):
        print(f" {k:28s} total={v['total']:6d} fake={v['fake']:6d} "

```

```

        f"real={v['total']-v['fake']:6d}")
    return out
# =====
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--root", default=os.path.join(ROOT, "data", "Datasets"))
    ap.add_argument("--check", action="store_true")
    ap.add_argument("--organize", action="store_true")
    ap.add_argument("--manifest", action="store_true")
    ap.add_argument("--mode", default="link", choices=["link", "symlink", "copy"])
    ap.add_argument("--out", default="")
    args = ap.parse_args()
    os.makedirs(args.root, exist_ok=True)
    if not any([args.check, args.organize, args.manifest]):
        args.check = args.manifest = True
    if args.check:
        check(args.root)
    if args.organize:
        organize(args.root, args.mode)
    if args.manifest:
        manifest(args.root, args.out)
if __name__ == "__main__":
    main()
/* ===== 文件: scripts/benchmark_cpu.py ===== */
"""部署路径的 CPU 吞吐基准 (ONNX Runtime) —— 申报书「CPU ≥ 8 张/秒」的验收脚本。
为什么要单独测 ONNX Runtime 而不是 PyTorch:
    申报书 3.1 的指标口径是「经模型封装与 CPU 加速优化后」的吞吐。
    实测同一台机器上, PyTorch 的 eager 推理比 ONNX Runtime 慢 5~8 倍
    (图优化 + 算子融合 + 线程池策略差异), 所以两个数字都要报,
    但验收应以本项目实际交付的推理路径 (ONNX / OpenVINO) 为准。
用法:
    python scripts/benchmark_cpu.py                # 默认测 deploy/lite*.onnx
    python scripts/benchmark_cpu.py --models deploy/lite_int8.onnx --reps 5
    python scripts/benchmark_cpu.py --threads 4      # 固定线程数, 减少波动
输出:
    outputs/cpu_benchmark.json
"""
from __future__ import annotations
import argparse
import json
import os
import platform
import statistics
import sys
import time
from typing import List
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
def _env_info() -> dict:
    try:
        import onnxruntime as ort
        ort_ver = ort.__version__
    except Exception: # noqa: BLE001

```

```

        ort_ver = "not installed"
    return {
        "platform": platform.platform(),
        "processor": platform.processor(),
        "cpu_count": os.cpu_count(),
        "onnxruntime": ort_ver,
        "torch": _torch_ver(),
    }
def _torch_ver() -> str:
    try:
        import torch
        return torch.__version__
    except Exception: # noqa: BLE001
        return "not installed"
def bench_onnx(path: str, shape=(1, 3, 224, 224), reps: int = 3, runs: int = 20,
               threads: int = 0, optimize: bool = True) -> dict:
    """多轮测量, 返回中位数与各轮明细。"""
    import numpy as np
    import onnxruntime as ort
    so = ort.SessionOptions()
    so.graph_optimization_level = (
        ort.GraphOptimizationLevel.ORT_ENABLE_ALL if optimize
        else ort.GraphOptimizationLevel.ORT_DISABLE_ALL
    )
    if threads > 0:
        so.intra_op_num_threads = threads
    sess = ort.InferenceSession(path, so, providers=["CPUExecutionProvider"])
    iname = sess.get_inputs()[0].name
    x = np.random.randn(*shape).astype(np.float32)
    for _ in range(5): # warmup
        sess.run(None, {iname: x})
    per_rep: List[float] = []
    for _ in range(reps):
        t0 = time.perf_counter()
        for _ in range(runs):
            sess.run(None, {iname: x})
        per_rep.append((time.perf_counter() - t0) / runs)
    med = statistics.median(per_rep)
    return {
        "model": os.path.basename(path),
        "size_MB": round(os.path.getsize(path) / 1e6, 1),
        "intra_op_threads": threads or "default",
        "reps_ms": [round(v * 1000, 1) for v in per_rep],
        "median_ms": round(med * 1000, 1),
        "median_fps": round(1.0 / med, 2),
        "best_fps": round(1.0 / min(per_rep), 2),
        "worst_fps": round(1.0 / max(per_rep), 2),
    }
def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--models", nargs="+",
                   default=["deploy/lite.onnx", "deploy/lite_simplified.onnx",

```

```

        "deploy/lite_int8.onnx"))
ap.add_argument("--image-size", type=int, default=224)
ap.add_argument("--reps", type=int, default=3, help="重复轮数, 取中位数以抗噪")
ap.add_argument("--runs", type=int, default=20, help="每轮推理次数")
ap.add_argument("--threads", type=int, default=0, help="0=默认(全部核心)")
ap.add_argument("--out", default="outputs/cpu_benchmark.json")
args = ap.parse_args()
env = _env_info()
print(f"[bench] {env['platform']}")
print(f"[bench] CPU 核心 {env['cpu_count']} / onnxruntime {env['onnxruntime']} / "
      f"输入 {args.image_size}x{args.image_size} / batch=1")
print(f"[bench] 每模型 {args.reps} 轮 x {args.runs} 次, 取中位数\n")
rows = []
for p in args.models:
    if not os.path.exists(p):
        print(f"[bench] 跳过 (不存在): {p}")
        continue
    r = bench_onnx(p, (1, 3, args.image_size, args.image_size),
                  args.reps, args.runs, args.threads)
    r["meets_cpu_8fps"] = r["median_fps"] >= 8
    rows.append(r)
    print(f"[bench] {r['model']}:<26s) {r['size_MB']}>6.1f) MB "
          f"中位 {r['median_ms']}>6.1f) ms {r['median_fps']}>6.2f) img/s "
          f"(最快 {r['best_fps']:.2f} / 最慢 {r['worst_fps']:.2f}) "
          f"{'达标' if r['meets_cpu_8fps'] else '未达标'})")
report = {"env": env, "image_size": args.image_size,
          "reps": args.reps, "runs_per_rep": args.runs, "rows": rows}
os.makedirs(os.path.dirname(os.path.abspath(args.out)) or ".", exist_ok=True)
with open(args.out, "w", encoding="utf-8") as f:
    json.dump(report, f, ensure_ascii=False, indent=2)
print(f"\n[bench] 已写入 {args.out}")

if __name__ == "__main__":
    main()

/* ===== 文件: scripts/smoke_test.py ===== */
"""端到端冒烟测试 —— 检查申报书里每一处公式对应的张量维度与梯度是否贯通
跑通即说明: 模型能前向、能反向、损失都参与优化、各模块维度与公式一致。
用法:
    python scripts/smoke_test.py          # 只做维度/梯度自检 (秒级)
    python scripts/smoke_test.py --full   # 额外生成演示数据 + 训练 1 轮 + 导出 ONNX
"""

from __future__ import annotations
import argparse
import os
import sys
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if ROOT not in sys.path:
    sys.path.insert(0, ROOT)
import torch # noqa: E402
from src.engine.trainer import optimization_parameters # noqa: E402
from src.losses.multi_task import MultiTaskLoss, ZScoreNormalizer # noqa: E402
from src.models.cs_cam import CSCAM # noqa: E402
from src.models.freq_branch import FrequencyBranch # noqa: E402

```

```

from src.models.gradient_stop import GradientStopLayer, grad_stop # noqa: E402
from src.models.mobile_unetv2 import EdgeHead, LocalizationBranch # noqa: E402
from src.models.spatial_branch import SpatialBranch # noqa: E402
from src.models.vib import HierarchicalVIB, beta_schedule # noqa: E402
from src.models.vibnet import build_model, load_config # noqa: E402
OK, BAD = "\033[32m[OK]\033[0m", "\033[31m[FAIL]\033[0m"
results = []
def check(name: str, cond: bool, detail: str = "") -> None:
    results.append((name, cond))
    print(f" {OK if cond else BAD} {name}" + (f" {detail}" if detail else ""))
# =====
def test_modules(cfg: dict) -> None:
    print("\n【1】单模块维度自检（对照申报书公式）")
    # ---- 1.1(1) 空域分支 ----
    print("\n 1.1(1) 空域特征提取分支 —— 公式 (1)(2)(3)")
    cfg["model"]["spatial"]["backbone"] = "tiny_vit"
    sb = SpatialBranch(cfg).eval()
    info = sb.sanity_check()
    for k, v in info.items():
        print(f" {k}: {v}")
    check("phi(x) 为 768 维 (公式1)", info["phi(x) 维度"][-1] == 768)
    check("F_spa 通道 896 (公式3)", info["F_spa 特征图"][1] == 896)
    check("F_spa 空间 14x14", info["F_spa 特征图"][2:] == (14, 14))
    check("F_spa(GAP) 896 维", info["F_spa(GAP) 向量维度"][-1] == 896)
    # ---- 1.1(2) 频域分支 ----
    print("\n 1.1(2) 频域特征提取分支 —— 公式 (4)-(12)")
    fb = FrequencyBranch(cfg).eval()
    finfo = fb.sanity_check()
    for k, v in finfo.items():
        print(f" {k}: {v}")
    check("A(u,v) 为 224x224 (公式5)", finfo["A(u,v)"] == (2, 1, 224, 224))
    check("P(u,v) 为 224x224 (公式6)", finfo["P(u,v)"] == (2, 1, 224, 224))
    check("F_freq 通道 256 (公式12)", finfo["F_freq 特征图"][1] == 256)
    # ---- 1.1(3) CS-CAM ----
    print("\n 1.1(3) 交叉注意力融合 —— 公式 (13)-(17)")
    cam = CSCAM(896, 256, 512).eval()
    cinfo = cam.sanity_check()
    for k, v in cinfo.items():
        print(f" {k}: {v}")
    check("F_fusion 为 (2,512,14,14) (公式17)", cinfo["F_fusion"] == (2, 512, 14, 14))
    # ---- 1.2 分层 VIB ----
    print("\n 1.2 分层 VIB —— 公式 (20)-(25)")
    vib = HierarchicalVIB(512, 256, 256).eval()
    vinfo = vib.sanity_check()
    for k, v in vinfo.items():
        print(f" {k}: {v}")
    check("mu 为 256 维 (公式20)", vinfo["mu"][-1] == 256)
    check("sigma 非负 (公式21)", vinfo["sigma_min/max"][0] >= 0)
    check("z 为 256 维 (公式23)", vinfo["z"][-1] == 256)
    check("KL 已裁剪到 <=10 (公式25)", vinfo["kl_clipped"] <= 10.0 + 1e-6)
    b = (beta_schedule(10), beta_schedule(30), beta_schedule(50))
    check("beta 退火分段正确 (公式24)",

```

```

        abs(b[0]) < 1e-9 and abs(b[1] - 0.05) < 1e-6 and abs(b[2] - 0.1) < 1e-6,
        f"beta(10),beta(30),beta(50)={b}" )
# ---- 1.3(1) 梯度停止 ----
print("\n 1.3(1) 梯度停止层")
v1 = GradientStopLayer.verify(stop=True)
v2 = GradientStopLayer.verify(stop=False)
check("stop=True 阻断梯度", v1["上游梯度全零"])
check("stop=False 恒等传递", v2["上游梯度全零"] is False)
# ---- 1.3(3) 定位支路 ----
print("\n 1.3(3) 定位支路 Mobile-Unetv2 —— 公式 (28)(29)")
loc = LocalizationBranch(cfg).eval()
with torch.no_grad():
    lo = loc(torch.randn(2, 512, 14, 14))
print(f"      mask_prob: {tuple(lo['mask_prob'].shape)} "
      f"edge_prob: {tuple(lo['edge_prob'].shape)} "
      f"参数量 {loc.num_millions():.2f} M")
check("M_pred 为 224x224 (公式28)", tuple(lo["mask_prob"].shape) == (2, 1, 224, 224))
check("M_edge_pred 为 224x224 (公式29)", tuple(lo["edge_prob"].shape) == (2, 1, 224, 224))
check("定位头参数量 < 5M", loc.num_millions() < 5.0, f"{loc.num_millions():.2f}M")
egt = EdgeHead.canny_edge(torch.zeros(2, 1, 224, 224))
sq = torch.zeros(2, 1, 224, 224)
sq[:, :, 50:150, 50:150] = 1.0
egt2 = EdgeHead.canny_edge(sq)
print(f"      全零掩码边缘像素={float(egt.sum()):.0f}, "
      f"方块掩码边缘像素={float(egt2.sum()):.0f}")
check("Canny 提取到边缘 (公式29)", float(egt2.sum()) > 0 and float(egt.sum()) == 0)
# =====
def test_end_to_end(cfg: dict) -> torch.nn.Module:
    print("\n【2】端到端前向 + 反向 (整图真伪 + 像素定位 + 边缘)")
    cfg["model"]["spatial"]["backbone"] = "tiny_vit"
    model = build_model(cfg)
    s = model.summary()
    for k, v in s.items():
        print(f"      {k:18s} {v:.3f}" if isinstance(v, float) else f"      {k:18s} {v}")
    info = model.sanity_check(batch=2)
    for k, v in info.items():
        print(f"      {k}: {v}")
    check("cls_logits (B,2) (公式27)", info["cls_logits (公式27)"] == (2, 2))
    check("mask_prob (B,1,224,224) (公式28)", info["mask_prob (公式28)"] == (2, 1, 224, 224))
    from src.deploy.optimize_openvino import estimate_model_size
    size_info = estimate_model_size(s["total (M)"])
    print("      体积核算: " + " ".join(
        f"{k}={v}" for k, v in size_info.items() if k.endswith("(MB)"))
    check("模型体积可满足 ≤120MB (INT8 量化后)", size_info["INT8达标"],
          f"FP32={size_info['FP32(MB)']}MB, "
          f"FP16={size_info['FP16(MB)']}MB, INT8={size_info['INT8(MB)']}MB"
          f" -> {size_info['建议']}")
# ---- 损失与反向 ----
print("\n【3】多任务损失 + 反向传播 (公式 26/29/30/31)")
model.train()
crit = MultiTaskLoss(cfg)
# ★ 必须用 Trainer 的同一份参数收集逻辑。以前这里写的是

```

```

# AdamW(model.trainable_parameters()), 漏掉了 criterion 里的  $\log \sigma$ ,
# 于是测试全绿、但真实训练里"可学习加权"从来没更新过。
opt_params, n_extra = optimization_parameters(model, crit)
opt = torch.optim.AdamW(opt_params, lr=1e-4)
x = torch.randn(4, 3, 224, 224)
y = torch.tensor([0, 1, 1, 0])
mask = torch.zeros(4, 1, 224, 224)
mask[1, :, 60:140, 60:140] = 1
mask[2] = 1
mvalid = torch.tensor([True, True, True, False])      # 第 4 个样本无掩码 (模拟混合批次)
out = model(x, sample_vib=True, beta=0.05)
losses = crit(out, y, mask, beta=0.05, active_tasks=["cls", "loc", "edge"],
              update_norm=True, mask_valid=mvalid)
print(f"      raw losses: {losses['raw']}")
print(f"      weights   : {losses['weights']}")
print(f"      skipped    : {losses['skipped']}")
check("四个损失项均参与计算 (公式31)", len(losses["raw"]) == 4)
# 不确定性加权的参数必须真的进了优化器, 否则权重会恒等于 0.5 (假"可学习")
uw = crit.weighting
check("不确定性加权参数已纳入优化器 (不是只收 model 参数)",
      n_extra > 0 and uw is not None and uw.log_var.requires_grad,
      f"额外纳入 {n_extra} 组  $\log \sigma$ ")
s_before = uw.log_var.detach().clone() if uw is not None else None
losses["total"].backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)      # 申报书 1.2(3) 梯度裁剪
grad_ok, no_grad = 0, 0
for n, p in model.named_parameters():
    if p.requires_grad:
        if p.grad is not None and torch.isfinite(p.grad).all():
            grad_ok += 1
        else:
            no_grad += 1
check("所有可训练参数梯度有效", no_grad == 0, f"{grad_ok} 个参数, {no_grad} 个异常")
opt.step()
check("优化器 step 未报错", True)
# 真正验证"可学习": 一步之后  $\log \sigma$  必须发生变化。
# 旧断言只检查权重 > 0 ( $0.5 * \exp(-s)$  恒为正), 所以永远为真、什么也没验证住。
if s_before is not None:
    moved = not torch.allclose(s_before, uw.log_var.detach())
    check("不确定性权重确实随梯度更新 (不再恒为 0.5)", moved,
          f" $\log \sigma$  变化量  $\max |\Delta| = \{ \text{float}((uw.log_var.detach() - s\_before).abs().max()) : .3e \}$ "
          if moved else " $\log \sigma$  一步未动, 说明未进优化器")
    w_now = crit.weights()
    print(f"       $\sigma$  after 1 step: "
          f"{ {k: round(float(v), 4) for k, v in w_now.items()} }")
# ---- 三阶段冻结策略 ----
print("\n【4】三阶段冻结/解冻策略")
for stage in cfg["train"]["stages"]:
    model.apply_stage(stage.get("freeze", []), stage.get("train", ["all"]))
    n = sum(p.numel() for p in model.parameters() if p.requires_grad) / 1e6
    print(f"      {stage['name']:24s} 可训练 {n:8.3f} M "
          f"freeze={stage.get('freeze')} train={stage.get('train')}")

```

```

        check(f"{stage['name']} 有可训练参数", n > 0)
    model.apply_stage([], ["all"])
    return model
# =====
def test_full_pipeline(cfg: dict) -> None:
    print("\n【5】完整流水线: 演示数据 -> 训练 -> 评测 -> ONNX 导出")
    from src.data.synth import generate_demo_dataset
    demo_root = os.path.join(ROOT, "data", "demo")
    if not (os.path.isdir(os.path.join(demo_root, "train", "image"))):
        info = generate_demo_dataset(demo_root, train=48, val=24, test=24)
        print(f"    演示数据集已生成: {info}")
    from src.data.datasets import build_data loaders
    from src.engine.trainer import Logger, Trainer
    from src.models.vibnet import build_model as bm
    sub = load_config(os.path.join(ROOT, "configs/default.yaml"))
    sub["model"]["spatial"]["backbone"] = "tiny_vit"
    sub["data"]["num_workers"] = 0
    sub["train"]["batch_size"] = 8
    loaders = build_data loaders(sub, use_demo=True)
    print(f"    train={len(loaders['train'].dataset)}    val={len(loaders['val'].dataset)}"
          f"    test={len(loaders['test'].dataset)}")
    for s in sub["train"]["stages"]:
        s["epochs"] = 1
    sub["train"]["log_every"] = 3
    sub["train"]["save_every"] = 999
    sub["project"]["output_dir"] = os.path.join(ROOT, "outputs", "smoke")
    sub["project"]["ckpt_dir"] = os.path.join(ROOT, "checkpoints")
    model = bm(sub)
    logger = Logger(os.path.join(sub["project"]["output_dir"], "smoke_log.txt"))
    trainer = Trainer(sub, model, torch.device("cpu"), loaders["train"], loaders["val"],
                      logger=logger, ckpt_prefix="smoke")
    hist = trainer.fit()
    check("三阶段训练可运行", len(hist) > 0, f"{len(hist)} 条记录")
    best = os.path.join(sub["project"]["ckpt_dir"], "smoke_best.pt")
    check("最佳权重已保存", os.path.exists(best), best)
    from src.evaluation.evaluate import evaluate_model, format_report
    res = evaluate_model(sub, best, "test", use_demo=True, device="cpu",
                         max_batches=3, save_dir=sub["project"]["output_dir"])
    print(format_report(res))
    check("评测流程可运行", "cls" in res and bool(res["cls"]))
    from src.deploy.export_onnx import export_onnx, simplify_onnx
    onnx_path = os.path.join(ROOT, "deploy", "smoke.onnx")
    try:
        export_onnx(best, onnx_path, 17, 224, os.path.join(ROOT, "configs/default.yaml"))
        check("ONNX 导出成功", os.path.exists(onnx_path),
              f"{os.path.getsize(onnx_path)/1e6:.1f} MB")
        p2 = simplify_onnx(onnx_path)
        print(f"    simplify -> {p2}")
    except Exception as e: # noqa: BLE001
        check("ONNX 导出成功", False, f"{type(e).__name__}: {e}")
    from src.deploy.optimize_openvino import convert_to_openvino, benchmark
    try:

```

```

    ir = convert_to_openvino(onnx_path, os.path.join(ROOT, "deploy", "smoke_ir"))
    if ir:
        benchmark(onnx_path, ir, 224, n_warmup=2, n_runs=5)
        check("OpenVINO 转换 + 性能测试", True)
    else:
        check("OpenVINO 转换 + 性能测试", False, "未安装 opencvino (非致命)")
except Exception as e: # noqa: BLE001
    check("OpenVINO 转换 + 性能测试", False, f"{type(e).__name__}: {e}")
# 统一推理封装 (桌面工具用的就是它)
try:
    from src.deploy.inference import Detector, list_available_backends
    print(f"    可用后端: {list_available_backends()}")
    det = Detector(backend="torch", ckpt=best,
                   config_path=os.path.join(ROOT, "configs/default.yaml"))
    sample = os.path.join(demo_root, "test", "image")
    any_img = sorted(os.listdir(sample))[0] if os.path.isdir(sample) else None
    if any_img:
        r = det.detect(os.path.join(sample, any_img))
        print(f"    Detector 推理: {r.summary()}")
        check("Detector 统一推理封装可用", 0.0 <= r.prob_fake <= 1.0)
except Exception as e: # noqa: BLE001
    check("Detector 统一推理封装可用", False, f"{type(e).__name__}: {e}")
# =====
def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--full", action="store_true", help="跑完整流水线 (含训练与导出)")
    ap.add_argument("--config", default=os.path.join(ROOT, "configs/default.yaml"))
    args = ap.parse_args()
    print("=" * 74)
    print("空频双分支轻量 VIB-Net —— 端到端冒烟测试")
    print("=" * 74)
    torch.manual_seed(3407)
    cfg = load_config(args.config)
    cfg["data"]["num_workers"] = 0
    cfg["train"]["batch_size"] = 4
    test_modules(cfg)
    test_end_to_end(cfg)
    if args.full:
        test_full_pipeline(cfg)
    n_ok = sum(1 for _, c in results if c)
    n_all = len(results)
    print("\n" + "=" * 74)
    print(f"测试结果: {n_ok}/{n_all} 通过")
    fails = [n for n, c in results if not c]
    if fails:
        print("未通过项: ")
        for n in fails:
            print(f"    - {n}")
    print("=" * 74)
    return 0 if n_ok == n_all else 1
if __name__ == "__main__":
    sys.exit(main())

```